ProbeFlow v20 독해 길잡이: 짧은 본문에서 깊은 근거로

이 길잡이는 새 실험의 보고가 아니라 문서를 읽는 순서를 정리한 설명이다. 본문은 질문·방법·핵심 결과와 한계를 짧게 연결한다. 보충자료는 실행 권한, 비용, 항등식의 유도와 실제 사례를 펼쳐 놓고, 뒤에 v19 원고 전체를 보존한다. 이 해설은 두 문서를 자기 말로 설명할 수 있도록 돕는다. 아래 길잡이 다음에는 기존 v19 해설의 모든 문장·표·수식이 그대로 이어진다. 짧은 본문에서 빠진 개발 연대기가 삭제된 것은 아니다.

독해 1. 세 문서를 어떤 질문으로 오갈까

처음에는 본문의 표 1과 Results를 읽고 "PS1보다 좋은가"와 "강한 TS1보다 좋은가"를 서로 다른 질문으로 적어 보자. 그다음 Method에서 네 평균 F·A·P·G의 정보 권한이 같은지 확인한다. oracle은 정답과 미획득 mask의 예측을 사후에 읽는다. 그래서 표의 작은 oracle 목적값은 실제 정책의 성공률이 아니다. 보충자료 S6과 S7은 바로 이 차이를 수식으로 추적한다.

지금 풀 질문	compact main	새 보충자료	이 해설에서 이어 읽을 곳
정책은 무엇을 보고 다음 센서를 고르나	Problem and Method	S1·S4	기존 3·5·9절
cap과 λ는 왜 둘 다 필요한가	Problem and Method	S2	기존 7절
root를 본 뒤 다른 센서를 읽는 것이 실제 온라인 측정인가	Experimental Setup	S3	기존 3·6절
threshold와 leaf는 누가 언제 고정했나	Experimental Setup	S5	기존 10절
F·A·P·G와 H·E·R은 무엇인가	Results	S6·S7	독해 2~4와 기존 13~16절
평균 오답이 줄었는데 완전 진단이 나빠진 이유는	Results	S10	독해 5와 기존 8·17절
어떤 문헌을 실제로 재현했나	Related Work	S12	기존 19절의 미재현 경계
무엇이 아직 증명되지 않았나	Limitations	S8·S11·S13	기존 20~22절

읽는 순서를 외우기보다 각 문서의 역할을 구분하자. 본문은 연구의 판단을 압축하고, 보충은 그 판단을 재구성하는 재료를 제공하며, 해설은 혼동하기 쉬운 말과 손계산을 풀어 준다. 보충 뒤의 옛 원고는 과거 질문과 실패가 지금의 판단으로 이어진 경로다. 오래된 판본에서 한 계산과 이번 판본에서 새로 한 계산을 합쳐 새 독립 실험처럼 세면 안 된다.

독해 2. 기호와 단위를 먼저 고정하기

F는 dev에서 이미 고른 고정 정책의 평균 목적값, G는 실제 gate가 낸 평균 목적값이다. A는 같은 cap 안의 모든 mask를 정답까지 보고 고르는 oracle 평균이고, P는 현재 두 leaf 안에서만 고르는 oracle 평균이다. H=F−A는 넓은 특권적 여지, E=P−A는 root·leaf·비용 제약을 함께 묶은 gap, R=G−P는 실제 분기와 특권적 선택의 차이다. 여기서 G를 제약 gap으로 쓰지 않는다.

$$H = F - A \geq 0, \quad E = P - A \geq 0, \quad R = G - P \geq 0$$

$$G - F = -H + E + R$$

H·E·R은 각각 비음수지만 최종 변화에서 H만 음수로 들어간다. E를 신경망의 순수 표현 오차, R을 관측값만으로 반드시 고칠 수 있는 학습 오류라고 부르면 틀린다. 정답을 아는 chooser와 실제 정책의 차이에는 정보 부족과 후보 제약이 섞인다. 또한 평균은 반드시 같은 k·가격표·cap·λ·평가 구간을 써야 한다. 서로 다른 설정의 숫자를 끼워 넣으면 항등식이 설명하는 대상부터 달라진다.
component error의 분모는 네 head 판단의 합인 4N이고, vector error의 분모는 사이클 N이다. quote는 선언한 원천 비용 장부다. 이 세 단위를 돈·초·성공률 하나로 바꾸지 않는다. equal quote에서는 원천 하나의 가격이 1이고 총 원천 가격은 3이므로 λ=1/20일 때 추가 원천 하나가 목적값에 더하는 값은 1/60이다. cap 2는 어느 사이클도 세 원천을 모두 살 수 없다는 별도 제약이다.

독해 3. 손계산 1: 평균 하나가 공개된 뒤에만 분기하기

실제 primary 사례 cycle 2184에서 먼저 TS1을 사서 공개된 평균은 35.442933333333336이다. 임계값은 35.446366666666667이며, 저장된 정확한 경계는 0x1.1b9228afdadcfp+5다. 평균이 임계값 이하

이므로 left로 가서 mask 5, 즉 TS1+PS1을 쓴다. mask 5는 센서 다섯 개가 아니다. 비트 1과 4가 켜진 원천 두 개다. 36.2273이 공개된 cycle 1785는 임계값 초과라 TS1만 쓰는 mask 4로 끝난다.

$$35.442933333333336 \le 35.44636666666667$$

cycle 2184의 감사 정답은 (100,90,0,90)이고 네 비교의 표시된 예측은 모두 (100,100,0,90)이다. 네 head 중 하나가 틀려 component 손실은 1/4다. gate는 원천 두 개, pair oracle은 TS1 하나를 써서 다음 차이가 난다.

$$u_{\mathrm{gate}} = \frac{1}{4} + \frac{2}{60} = \frac{17}{60}$$

$$u_{\mathrm{pair}} = \frac{1}{4} + \frac{1}{60} = \frac{4}{15}$$

차이는 1/60이다. 이 사이클에서는 예측 오류를 줄이지 못한 추가 구매다. 그러나 이렇게 고른 사례 몇 개만 보고 전체 빈도를 계산하지 않는다. 보충 S9·S10의 10개 사례는 일부를 결과에 따라 선택한 설명용 기록이며, 성능은 421개 전체 평가 사이클의 집계로 판단한다. 또한 TS1 평균은 이미 끝난 60초 기록의 평균이다. 사이클 시작 시점의 조기 경보를 입증하는 계산이 아니다.

독해 4. 손계산 2: pair oracle과 강한 TS1 경계

primary TS1/k1은 421사이클에서 798개 component 오류와 총 quote 421을 갖는다. component 분모는 1684다. 원천 하나를 매번 샀으므로 평균 비용 벌점은 1/60이다.

$$\frac{798}{1684} + \frac{1}{60} = \frac{12391}{25260} = P$$

실제 gate는 같은 798개 오류에 quote 443을 쓴다. 따라서 목적값은 12413/25260이고, pair oracle보다 큰 양은 다음과 같다.

$$G - P = \frac{12413 - 12391}{25260} = \frac{22}{25260} = \frac{11}{12630}$$

왜 이것이 단순한 동점보다 강한가? TS1 mask 4는 두 leaf 중 하나이므로 매 사이클 pair oracle은 TS1보다 나쁠 수 없다. 그 차이들이 모두 비음수인데 평균 차이가 0이면 각각도 0이어야 한다. 그러므로 같은 두 leaf를 쓰는 어떤 분기든 이 유한 평가 구간의 같은 손실에서 TS1을 엄격히 이길 수 없다. 보충 S7은 무작위 분기에도 같은 하한이 적용되는 이유와, 22개 left branch의 추가 비용이 전체 gap을 설명하는 정수 단위 논증을 보인다.
이 결론은 현재 leaf·예측기·가격·손실·표본에 묶여 있다. 미래 자료에서 모든 적응이 무의미하다는 정리가 아니다. leaf나 예측기를 바꾸면 새 개입이므로 별도 평가가 필요하다. 반대로 지금의 두 leaf를 유지하면서 threshold만 더 잘 찾으면 반드시 TS1을 이긴다는 기대는 이 경계와 맞지 않는다. PS1 대비 78개 오류 감소를 곧바로 "적응의 순효과"라고 부르지 않는 이유다.

독해 5. 손계산 3: 평균 진단과 완전 진단의 교환

secondary는 λ=0이라 비용 벌점 없이 component 오류 수로 바로 계산할 수 있다. 고정·전체 oracle·두 leaf oracle·실제 gate의 오류 수는 차례로 924·625·789·861이다.

$$-(924 - 625) + (789 - 625) + (861 - 789) = -299 + 164 + 72 = -63$$

모두 1684로 나누면 실제 평균 차이는 $-63/1684$다. 하지만 vector 오류는 381에서 391로 10개 늘고 quote는 842에서 518로 줄었다. 네 판단 중 일부를 더 잘 맞히면서 네 판단 전부를 맞힌 사이클은 줄 수 있다. 그래서 "오류 63개 감소"는 "완전 진단 성공 63개 증가"가 아니다. 예를 들어 cycle 1788에서는 gate가 싼 PS1만 쓰고 head 하나를 틀리지만, 고정 PS1+FS1은 네 head를 모두 맞힌다.

독해 6. 자주 틀리는 해석을 스스로 고치기

틀린 해석	고쳐 말하기
390 gate와 1170행이면 표본이 충분히 커졌다	같은 한 rig·625사이클의 종속 설정과 반복 집계다
fit 밖 392/421이면 그 392개는 모두 오답이다	root의 fit min/max 밖이라는 관측 진단이며 정답 여부나 밀도 지지가 아니다
fit 목표가 한 tuple이면 관측 학습은 수학적으로 불가능하다	같은 목표 안에서도 오류 관계를 배울 수 있으나 새 조건 전이는 검증되지 않았다
R이 양수면 학습으로 그만큼 개선할 수 있다	정답·미관측 예측을 아는 oracle과의 gap이며 관측만으로 달성 가능한지는 별도다
이번 검산기가 분석 함수를 import하지 않았으니 외부 재현이다	대체 산술 구현과 별도 root 검토가 있었지만 새 독립 경험적 재현은 아니다
새 v20 문서가 나왔으니 새 모델과 센서 실험이 있었다	이번은 기존 결과의 compact 조판·심화 유도·독해 재구성이다

마지막으로 책을 덮고 여섯 문장을 직접 설명해 보자. "정책은 어떤 값만 볼 수 있는가?", "cap과 λ의 역할은 어떻게 다른가?", "G와 E는 왜 다른 기호인가?", "pair=TS1이 threshold 탐색에 주는 경계는 무엇인가?", "component −63과 vector +10이 어떻게 공존하는가?", "독립 장치와 실제 시간·실패 기록이 왜 다음 실험에 필요한가?" 답이 막히는 항목만 표의 보충 절과 기존 해설로 되돌아가면 된다.

ProbeFlow v19 심화 해설: 원리·구현·실패 원인 분해

새 분석을 이해하기 전에 데이터·관측 권한·예측기·비용·정책·평가의 관계를 한 번에 설명하기 위한 로컬 심화 해설이다. 아래 source URL은 비교 문헌이며 이 해설의 외부 출판 주소가 아니다.

1. ProbeFlow는 대체 무엇을 고르는 연구인가

공장 설비를 진단한다고 생각하자. 모든 센서를 늘 읽으면 편하지만 측정·전송·계산 비용이 생긴다. ProbeFlow의 질문은 '분류기를 더 크게 만들까'보다 먼저 '이미 가진 정보로 끝낼까, 어떤 정보를 더 살까'다. 정보 획득 정책과 마지막 진단기를 분리해야 이 질문에 답할 수 있다.

설명용으로 병원의 검사 순서를 떠올릴 수 있지만 현재 자료는 의료 데이터가 아니다. 실제 로컬 연구는 한 유압 장비의 이미 기록된 60초 사이클을 사용한다. 공개된 센서값에 따라 추가 요청이 달라지는 코드는 존재한다. 그러나 장비에서 새 센서를 켜거나 같은 과거 60초를 뒤늦게 측정한 실험은 아니다.

연구가 궁극적으로 보여야 할 것은 같은 정보·같은 예산·같은 진단 기준에서 관측에 따른 선택이 단순한 고정 센서보다 도움이 되는지다. 현재 v19는 그 우월성을 주장하기보다, 저장된 자료에서 개선 여지와 실패 원인을 더 정밀하게 분해한다.

그림으로 이해하기: 자료·정책·진단·감사를 서로 다른 상자로 읽기

직접 작성한 교육용 구조도다. 아래 연결은 구현의 정보 권한을 설명하며 물리 장비 실험이나 원문 그림의 재현이 아니다.

정책이 센서를 고를 때와 감사자가 정답으로 채점할 때의 정보 권한이 다르다.

자료·정책·진단·감사를 서로 다른 상자로 읽기

- 비공개 자료 → 획득 경계: 가려 둔 값
- 획득 정책 → 획득 경계: 합법 요청
- 획득 경계 → 동결 진단기: 공개 특징
- 동결 진단기 → 사후 평가: 예측만

2. 처음 읽을 때 헷갈리는 말부터 정리하기

말	여기서의 뜻	혼동하면 생기는 오류
cycle	이미 기록된 60초 단위 한 행	625개를 수십만 독립 실험으로 세기
source	PS1·FS1·TS1 센서 원천	원천 하나와 특징 하나를 같은 가격으로 세기
feature	원천에서 계산한 평균·표준편차·기울기	raw 원천을 안 읽고 파생 통계만 무료로 쓰기

mask	획득한 원천들의 집합을 나타내는 정수	mask 5를 센서 다섯 개로 해석하기
predictor	공개 특징으로 네 진단을 내는 동결 예측기	gate를 바꾸면서 진단기도 바꾸고 효과를 혼합하기
policy / gate	무엇을 읽고 멈출지 정하는 규칙 / 작은 관측 분기	단순 분기를 Bellman 신경망 재현이라고 부르기
quote / cap	선언한 비용 proxy / 사이클별 지출 상한	quote를 측정된 원화·전력·지연으로 해석하기
oracle	정답과 미관측 예측을 써서 계산한 사후 하한	현장에서 사용할 수 있는 정책으로 제시하기

head는 네 진단 목표 중 하나다. cooler·valve·pump·accumulator에 대한 판단을 따로 세는 component 오류와 네 개를 한꺼번에 맞췄는지 보는 vector 오류는 다른 질문이다. '정확도'라는 한 단어로 섞지 않는다.

3. 센서 원천 한 번과 특징 세 개는 어떤 관계인가

현재 원천은 압력 계열 PS1, 유량 계열 FS1, 온도 계열 TS1이다. 저장 파일의 한 사이클에는 각각 6,000·600·60개 표본이 있다. 정책이 읽는 것은 그중 원천 평균이다. 마지막 진단기는 획득한 원천의 평균·모표준편차·기울기 특징을 사용할 수 있다. 즉 정책의 분기 입력과 진단기의 특징 입력을 구분한다.

$$\bar{x}_{i,j} = \frac{1}{T_j} \sum_{t=1}^{T_j} x_{i,j,t}$$

$$(T_{\mathrm{PS1}}, T_{\mathrm{FS1}}, T_{\mathrm{TS1}}) = (6000, 600, 60)$$

i는 사이클, j는 원천, t는 한 원천 안의 표본 번호다. 이 값은 이미 완성된 사이클의 평균이며 사건 이전에 이용 가능한 조기 관측이라는 뜻이 아니다.

원천 획득과 파생 특징 공개가 어떤 비용 계약을 갖는지 먼저 정해야 한다. 현재 원천 수준 실험은 미리 정한 특징 묶음을 사용한다. 더 일반적인 ProbeFlow 아이디어의 공유 작업 DAG에서는 부모 읽기 비용을 한 번만 청구하고 자식 계산 비용을 별도로 둘 수 있다. 그 일반 설계를 이번 세 원천 실측에서 모두 구현한 것처럼 쓰지 않는다.

그림으로 이해하기: 원천 읽기·공개 평균·마지막 특징의 경계

직접 작성한 교육용 구조도다. 아래 연결은 구현의 정보 권한을 설명하며 물리 장비 실험이나 원문 그림의 재현이 아니다.

원천 파일을 읽었다는 사실만으로 정책에 모든 값과 정답을 넘길 권한이 생기지 않는다.

원천 읽기·공개 평균·마지막 특징의 경계

- PS1 원천 → 공개 평균: 획득 뒤 공개
- PS1 원천 → 특징 묶음: 선언한 특징
- 공개 평균 → 정책의 다음 선택: 관측값
- 특징 묶음 → 동결 예측기: 허용 좌표

4. mask 0부터 7까지 직접 읽기

mask	이진 표기	허용 원천
0	000	없음
1	001	PS1
2	010	FS1
3	011	PS1 + FS1
4	100	TS1
5	101	PS1 + TS1
6	110	FS1 + TS1
7	111	PS1 + FS1 + TS1

정수 5는 이진수 101이므로 PS1과 TS1을 뜻한다. 센서 다섯 개가 아니다. empty mask 0도 가능한 후보이며 별도로 정의된 fallback 예측을 사용한다. 마스크가 커진다는 말은 정수 크기가 아니라 집합 포함 관계를 뜻해야 한다.
모든 정보를 넣으면 항상 더 정확할까? 최적 베이즈 결정자가 정보를 무시할 수 있다면 정보가 늘어나는 것이 불리할 이유가 없다. 하지만 현재 동결된 최근접 이웃 규칙은 좌표가 늘면 이웃과 투표가 바뀐다. 불필요한 좌표를 자유롭게 무시하는 최적화가 없으므로 부분 원천보다 전체 원천의 실측 오류가 클 수 있다. 정보의 이상적 가치와 특정 부정확한 예측기의 출력을 구별해야 한다.

5. k와 예측 캐시는 무엇이며 왜 동결하는가

k는 이웃 투표에 쓰는 이웃 수다. 기존 v15는 학습 bank와 train-only 표준화 정보를 유지한 채 head별 class-balanced kNN의 k=1,3,5,11,31과 여덟 mask에 대한 예측을 만들었다. 현재 gate 연구는 이 중 k1과 k3을 비교하며 예측 캐시를 재사용한다. v19에서 새 예측기를 적합하거나 새 forward를 실행한 것은 아니다.
캐시의 키는 (k, mask, cycle)이고 값은 네 목표의 예측 tuple이다. 정책이 멈춘 뒤에만 해당 mask의 예측을 꺼낸다. 최종 점수를 계산하는 감사자는 다른 mask의 예측도 볼 수 있지만, 실행 정책이 이 표 전체를 미리 보고 다음 원천을 고르면 미관측 정보 누출이다.
동결은 약점을 감추는 장치가 아니라 효과를 분리하는 장치다. 정책 A와 B가 서로 다른 진단기를 쓰면 차이가 센서 선택 때문인지 예측기 때문인지 알기 어렵다. 먼저 같은 k·학습 bank·예측 캐시로 획득 정책만 대조한 다음, 더 강한 예측기와의 교차 비교를 별도로 설계한다.

6. 네 단계의 시간 분할을 한 장으로 보기

역할	cycle ID	개수	이 구간으로 하는 일
predictor train	1..1540	1540	이미 동결된 예측기 bank
policy fit	1561..1662	102	threshold·leaf 후보 작성
policy dev	1663..1764	102	작성된 후보 중 정책 선택
exposed evaluation	1785..2205	421	고정 정책 채점·사후 원인 분석

각 구간 사이의 제외 사이클은 분할 계약의 일부다. 제외 간격이 있다는 이유만으로 독립 장비·독립 고장 사건·독립 통계 표본이 되는 것은 아니다. 또한 evaluation은 과거 연구에서 이미 봤다. 이번 실행에서 마지막에 파일을 열었다는 사실은 과거 노출을 지우지 못한다.

그림으로 이해하기: 학습·선택·감사의 방향

직접 작성한 교육용 구조도다. 아래 연결은 구현의 정보 권한을 설명하며 물리 장비 실험이나 원문 그림의 재현이 아니다.
정책을 평가한 뒤 더 잘 나온 설정으로 주분석을 바꾸지 않는다.
학습·선택·감사의 방향

- 진단기 bank → 정책 후보 작성: 동결 예측
- 정책 후보 작성 → 정책 잠금: 후보만
- 정책 잠금 → 노출 평가 채점: 선택 고정
- 노출 평가 채점 → v19 원인 진단: 사후 분석

7. 비용 항과 hard cap을 동시에 쓰는 이유

$$C(m) = \sum_{j \in m} c_j, \qquad \mathcal{F}_B = \{m : C(m) \le B\}$$

$$u_i(m) = \ell_i(m) + \lambda \frac{C(m)}{\sum_j c_j}$$

$$U(m) = \frac{1}{n}\sum_{i=1}^{n} u_i(m)$$

c_j는 원천 quote다. C는 원천 비용의 합이며 이 연구의 proxy다. λ는 정확도와 비용의 교환 비율을 선언하는 계수다. cap은 합법 행동을 제한하고 λ는 그 안에서 무엇을 선호할지 정한다.
equal quote에서는 원천당 1, 세 원천 합은 3이다. λ=0.05이면 원천 하나의 정규화 비용 항은 0.05/3이다. cap2를 지킨 두 정책도 한 정책은 매번 두 개를 읽고 다른 정책은 일부에서 한 개만 읽을 수 있어 평균 지출은 다르다. 같은 cap과 같은 평균 비용은 같은 말이 아니다.
raw quote는 원천당 표본 수를 반영한 6000·600·60의 proxy다. equal quote와 단위가 다르므로 421이라는 숫자를 그대로 다른 가격표와 비교하지 않는다. 실제 원화·전력·네트워크 지연을 측정한 실험이 아니며, 이미 전체 데이터를 저장하는 데 든 비용을 정책 replay가 절감했다는 뜻도 아니다.

8. component와 vector가 서로 반대 결론을 내는 이유

$$\ell_i(m) = \frac{1}{4}\sum_{h=1}^{4} 1\{\hat{y}_{i,h}(m) \neq y_{i,h}\}$$

$$E_{\text{comp}} = \frac{\sum_{i,h} 1\{\hat{y}_{i,h} \neq y_{i,h}\}}{4n}$$

$$E_{\text{vec}} = \frac{1}{n}\sum_{i} 1\{\hat{\mathbf{y}}_i \neq \mathbf{y}_i\}$$

i는 사이클이고 h는 네 진단 목표다. evaluation의 두 분모는 각각 1684와 421이다. 지표를 선택한 뒤 결과에 맞게 바꾸면 안 된다.
교육용 반례를 직접 세자. 두 사이클에서 A의 오답 head 수가 (4,0), B가 (1,1)이면 A의 component 오류는 4/8, vector 오류는 1/2다. B는 component 2/8로 좋아지지만 vector 2/2로 나빠진다. 오답을 한 사이클에 몰아넣느냐 여러 사이클로 퍼뜨리느냐가 다르기 때문이다.

그림으로 이해하기: 판단의 개수와 완전 진단의 개수
직접 만든 두 사이클 교육용 반례이며 유압 자료나 외부 논문 실험 결과가 아니다.
평균 head 오류 개선은 완전한 진단 개선을 보장하지 않는다.
교육용 두 사이클 · component

대상	정책: 0=A, 1=B	오류율
교육용 계산	0	0.5
교육용 계산	1	0.25

교육용 두 사이클 · vector

대상	정책: 0=A, 1=B	오류율
교육용 계산	0	0.5
교육용 계산	1	1

9. gate의 실행을 의사코드 대신 말로 따라가기

고정 정책은 처음부터 최종 mask를 정한다. gate는 먼저 root 원천 하나를 사고, 공개된 평균이 threshold 이하인지 확인해 두 leaf mask 중 하나를 정한다. 그 leaf에 아직 획득하지 않은 원천이 있으면 추가로 읽고, 없으면 STOP한다. 현재 gate leaf는 root를 포함하고 최대 두 원천을 쓴다.

순서	허용되는 입력	결정
1	원천 목록·가격·cap·선택된 정책	root를 합법적으로 요청
2	공개된 root 평균	threshold 이하/초과 분기
3	선택 leaf와 현재 acquired mask	누락 원천만 추가 요청
4	종료 mask에 해당하는 특징/캐시	네 목표 예측
5	이제 감사자에게만 정답 공개	오류·quote·oracle 진단

주분석 강제 gate의 root는 TS1이다. 어느 구간에서는 TS1로 멈추고 다른 구간에서는 PS1을 더해 mask5가 된다. root=TS1이라고 해서 다른 센서의 값을 이미 알고 분기하는 것이 아니다. v17의 hidden-value poison fixture는 이 코드 경계를 점검했지만 OS 수준 보안 격리나 모든 미래 구현의 무누출을 인증하지는 않는다.

10. threshold·leaf·정책 선택은 각각 언제 일어나는가

fit에서 root 평균의 사분위 위치로 threshold 후보를 만든다. 각 branch에 필요한 최소 개수를 확인하고, 비용과 합법 mask 조건을 지키는 서로 다른 두 leaf를 fit 목적값으로 고른다. dev에서는 이렇게 작성된 fixed와 gate 후보의 오류·비용을 비교한다. 같은 목적값이면 낮은 quote, 단순 fixed 우선 등 이미 선언한 동률 규칙을 적용한다.

best_fixed는 합법 고정 mask 전부를 비교한다. gate_only는 gate가 있으면 gate만 고른다. adaptive_enabled는 fixed와 gate의 합집합에서 고른다. 합집합의 dev 최저값이 fixed-only보다 나쁘지 않은 것은 후보 포함 관계의 산술이다. evaluation에서도 낫다는 경험 법칙이나 정리가 아니다. 한 설정에서 gate 후보가 없으면 fallback을 그대로 기록한다. gate를 강제로 사용하는 family가 더 복잡하다는 이유만으로 우월하다고 해석하지 않는다. 현재 gate는 최대 두 원천, fixed는 cap이 허용하면 세 원천 모두를 쓸 수 있다. 정책 class의 포함 관계를 이름만 보고 짐작하지 않는다.

11. 기존 결과를 가장 강한 대조군까지 읽기

equal cap2 조건	고정 대비 gate component	vector 변화	quote 합 변화
k1·λ0.05 주분석 union	876→876 /1684	391→391 /421	421→421
k1·λ0.05 강제 gate vs PS1	876→798 /1684	391→391 /421	421→443
같은 강제 gate vs 역사적 TS1	798→798 /1684	391→391 /421	421→443
k3·λ0 gate vs PS1+FS1	924→861 /1684	381→391 /421	842→518
k3·λ0.05 gate vs PS1	799→861 /1684	391→391 /421	421→518

두 번째 줄만 보면 gate가 뚜렷이 좋다. 세 번째 줄을 붙이면 같은 오류의 TS1이 더 싸다. TS1은 과거 선택 접근이 다른 역사적 대조군이므로 새로운 dev 선택에 몰래 넣어 우승자로 바꾸지는 않는다. 그렇다고 이미 있는 더 강한 반례를 결과에서 지우지도 않는다.

k3에서는 목표가 갈린다. λ0의 component 이득은 실제 계산된 로컬 결과지만 vector는 나빠진다. λ0.05에서는 dev의 작은 우위로 고른 gate가 evaluation에서 더 틀리고 더 쓴다. 따라서 '복잡한 적응형이 항상 이긴다'도 '적응형은 쓸모없다'도 현재 데이터가 뒷받침하지 않는다.

12. fit 정답이 한 종류뿐이면 무엇이 부족한가

구간	서로 다른 정답 tuple	fit에서 본 tuple 수	fit에서 못 본 tuple 수
fit102	1	102	0
dev102	11	2	100
evaluation421	38	30	391

여기서 서로 다른 tuple 개수와 개별 사이클 개수를 나눠 읽어야 한다. evaluation의 tuple 종류 38개와 미관측 tuple을 가진 사이클 391개는 같은 수량이 아니다. 앞 102개가 모두 (100,100,0,90)이면 그 구간은 다양한 고장 조합에서 어떤 센서가 유용한지 학습할 대비를 충분히 제공하지 못한다.

root 평균이 fit의 최솟값·최댓값 밖인지도 v19에서 따로 진단한다. 범위 안이라고 충분한 밀도 지원이 있다는 뜻은 아니고, 범위 밖이라고 반드시 오답이라는 뜻도 아니다. 정답 tuple 지원과 관측값 범위 지원은 서로 다른 축이다. 비어 있는 strata의 평균은 0이 아니라 null로 남긴다.

13. v19: 좋아질 여지와 그 여지를 못 쓴 원인을 분리하기

평균 오류 하나만으로는 gate가 실패한 이유를 모른다. 같은 예측기·가격·cap에서 어떤 합법 mask를 골라도 나아질 수 없었을 수도 있다. 더 나은 mask는 있지만 현재 gate의 두 leaf에는 없을 수도 있다. 정답을 아는 특권 oracle은 더 좋은 leaf를 골랐더라도, 그 차이를 현재 공개 관측으로 알아낼 단서가 있었는지는 아직 검증되지 않았다. v19는 이 차이를 정확한 산술로 나누며 관측 가능한 개선량으로 동일시하지 않는다.

이 작업은 이미 작성된 gate를 다시 학습하지 않는다. 390개 gate-context 기록을 625개의 기존 사이클에서 진단한다. k와 가격표·cap·λ를 고정한 채 비교하고 정답을 쓰는 oracle은 사후 감사에서만 계산한다. oracle로 정책을 다시 고르거나 새 배포 성능을 보고하지 않는다.

14. 두 oracle을 고르면 원인이 보인다

$$a_i = \min_{m \in \mathcal{F}_B} u_i(m)$$

$$b_i = \min\{u_i(m_L), u_i(m_R)\}$$

$$a_i \le b_i \le u_i(m_{\mathrm{gate},\,i})$$

별표는 현장에서 정답을 아는 결정이 아니라 사후 하한이다. 두 leaf 모두 cap 안에 있고 root를 포함한다. 같은 k·가격·λ를 공유해야 포함 관계와 부등식이 성립한다.

그림으로 이해하기: 실제 정책과 두 사후 oracle의 권한

직접 작성한 교육용 구조도다. 아래 연결은 구현의 정보 권한을 설명하며 물리 장비 실험이나 원문 그림의 재현이 아니다.

실행 규칙은 위쪽, 정답을 쓰는 원인 진단은 아래쪽이다. 아래 정보를 정책 입력으로 올리지 않는다.

실제 정책과 두 사후 oracle의 권한

- 실제 공개 평균 → 실제 gate: 관측만
- 실제 gate → 실제 손실: 실행
- 전체 mask oracle → 원인 분해: 특권 비교
- 두 leaf oracle → 원인 분해: 특권 비교
- 실제 손실 → 원인 분해: 실제값

15. 핵심 항등식을 직접 유도하기

$$H = F - A \ge 0, \quad E = P - A \ge 0, \quad R = G - P \ge 0$$

$$G - F = -(F - A) + (P - A) + (G - P) = -H + E + R$$

F는 잠긴 고정 정책의 평균 목적값, A는 전체 oracle 평균, P는 두 leaf oracle 평균, G는 실제 gate 평균이다. 같은 표본과 목적함수에서 중간항을 더하고 빼는 정확한 항등식이지 통계적 추정 보장이 아니다.

H는 현재 고정 정책에서 더 좋아질 수 있는 특권적 여지다. E는 현재 root·두 leaf·비용 제약 묶음으로 표현하지 못한 양이며 순수한 신경망 표현 오차만을 뜻하지 않는다. R은 두 leaf 안에서 실제 분기와 정답을 아는 chooser 사이의 차이다. 관측 정보 부족·정책 제약·동률 선택도 섞여 있으므로 순수 학습 오류나 인과 효과로 단정하지 않는다. 세 양은 비음수지만 H 앞에는 마이너스가 붙는다. H보다 E+R이 크면 gate가 고정보다 나빠진다.

고정 정책 F가 그 gate의 두 leaf에 들어 있지 않아도 F는 전체 합법 mask에 들어 있으므로 H는 비음수다. 반대로 pair oracle P가 고정 F보다 항상 낫지는 않다. 그래서 쌍 oracle이 고정 정책조차 이기

지 못하면 그 leaf쌍의 라우팅만 바꿔 해당 표본·목표에서 엄격히 이길 수 없다는 조건부 결론을 낼 수 있다.

16. 실제 v19 주분석: 이 두 leaf로 TS1을 이길 수 있는가

같은 421개 평가·k1·equal cap2·λ0.05	평균 목적값의 정확한 값
고정 PS1 F	13561/25260
전체 oracle A	10201/25260
두 leaf oracle P	12391/25260
실제 gate G	12413/25260

$$G - F = -\frac{3360}{25260} + \frac{2190}{25260} + \frac{22}{25260} = -\frac{1148}{25260}$$

모두 component 오류와 정규화 quote를 합친 같은 목적 단위다. 오류 개수나 percentage point와 그대로 섞지 않는다. 등식 양변의 분모를 맞춰 계산하면 25260에서 −3360+2190+22=−1148이다.
이 gate는 TS1만 쓰는 mask4와 TS1+PS1을 쓰는 mask5 중에서 고른다. 두 leaf oracle의 목적값은 역사적 고정 TS1과 정확히 같다. 정답을 아는 특권적 chooser도 같은 평가 기록에서 TS1보다 낮은 평균 목적값을 만들지 못한다. 따라서 현재 두 leaf를 유지하고 threshold만 더 잘 고르는 것으로는 이 특정 TS1 대조를 엄격히 이길 수 없다.
실제 gate는 22사이클에서 추가 원천을 요청했고, component 오류는 TS1과 같은 798이다. 22개의 추가 quote가 정규화 비용으로만 남아 routing regret 22/25260이 된다. 이 결론은 현재 동결 예측·두 leaf·421사이클·비용 가정에 묶여 있다. 다른 예측기·새 leaf·다른 장비·다른 손실까지 적응형을 부정하지 않는다.

17. 실제 v19 보충 사례: 여지는 있지만 분기에서 놓친 양

사전 규칙으로 고른 k3·equal cap2·λ0의 gate에서는 고정 정책의 component 오류 924가 전체 mask oracle에서 625, 두 leaf oracle에서 789, 실제 gate에서 861이 된다. 즉 −299+164+72=−63이며 모두 같은 1684개 head-cycle 판단을 분모로 한다. 숫자가 어떻게 생겼는지 단계를 밟으면 단순한 'gate 이득 63'보다 원인이 잘 보인다.

그림으로 이해하기: 실제 기록에서 세 원인과 관측 결과를 함께 읽기

v19에서 기존 k3 예측을 새로 분석한 실제 로컬 수치다. 하나의 이미 노출된 장비 자료이며 oracle은 정답을 사용하는 사후 하한이다. 외부 논문 성능이나 새 예측 결과가 아니다.
순이득이 있어도 표현 gap164와 라우팅 손실72가 남는다. 어느 문제를 고칠지 분리할 수 있다.
k3·λ0 · 단계별 component 오류

대상	0=고정, 1=전체 oracle, 2=쌍 oracle, 3=gate	오류 수 /1684
저장 예측에서 새로 계산	0	924
저장 예측에서 새로 계산	1	625
저장 예측에서 새로 계산	2	789
저장 예측에서 새로 계산	3	861

k3·λ0 · 분해한 변화량

대상	0=−여지, 1=표현 gap, 2=라우팅, 3=순변화	오류 수 변화 /1684
정확한 유한 산술	0	-299
정확한 유한 산술	1	164
정확한 유한 산술	2	72
정확한 유한 산술	3	-63

이 사례는 component 목표에서의 개선 여지를 말한다. 같은 gate의 완전 진단 오류는 381에서 391로 늘고 quote는 842에서 518로 줄었다. 따라서 component 분해가 좋아졌다고 vector의 안전성이 좋아졌다고 결론내리지 않는다. 어떤 실패를 더 중요하게 볼지 연구 목표와 실제 사용자 손실에서 먼저 정해야 한다.

18. 실제 사이클을 추적하는 읽기 절차

해설의 다음 표는 v19의 traced_cases.json에 고정한 두 정책 각각의 다섯 사례다. 첫 평가 사이클, 다른 branch, 양의 routing regret 또는 pair gap 등 미리 선언한 설명용 규칙으로 고르며 일부는 결과에 따라 뽑힌 사후 사례다. 이 열 행으로 성능 빈도나 성공률을 추정하지 않는다.

행을 읽을 때는 먼저 k·가격·cap·λ가 같은지 확인한다. 그다음 root 평균과 threshold를 비교해 선택 leaf를 종이에 쓴다. 요청 원천·총 quote·네 목표 예측을 확인하고 마지막에 정답을 공개한다. 전체 oracle·쌍 oracle의 값은 진단 이후에만 대조한다. 이 순서를 거꾸로 읽으면 정답을 알고 행동을 골라도 되는 것처럼 느끼기 쉽다.

primary_forced_gate · g129: k=1, cap=2, λ=1/20

cycle 1785 · 설명 사례 선택: first_evaluation_cycle. 공개 TS1 평균=36.2273, threshold=35.446367, branch=right. 감사 정답=[100, 90, 0, 130]. fit tuple 지원=False, root fit 범위 밖=True.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	1	[100, 100, 0, 90]	2	1	31/60
실제 gate	4	[100, 100, 0, 90]	2	1	31/60
두 leaf oracle	4	[100, 100, 0, 90]	2	1	31/60
전체 oracle	1	[100, 100, 0, 90]	2	1	31/60

cycle 2184 · 설명 사례 선택: first_left_leaf. 공개 TS1 평균=35.442933, threshold=35.446367, branch=left. 감사 정답=[100, 90, 0, 90]. fit tuple 지원=False, root fit 범위 밖=False.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	1	[100, 100, 0, 90]	1	1	4/15
실제 gate	5	[100, 100, 0, 90]	1	2	17/60
두 leaf oracle	4	[100, 100, 0, 90]	1	1	4/15
전체 oracle	1	[100, 100, 0, 90]	1	1	4/15

cycle 1786 · 설명 사례 선택: first_right_leaf. 공개 TS1 평균=36.212483, threshold=35.446367, branch=right. 감사 정답=[100, 90, 0, 130]. fit tuple 지원=False, root fit 범위 밖=True.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	1	[100, 100, 0, 90]	2	1	31/60
실제 gate	4	[100, 100, 0, 90]	2	1	31/60
두 leaf oracle	4	[100, 100, 0, 90]	2	1	31/60
전체 oracle	1	[100, 100, 0, 90]	2	1	31/60

cycle 2185 · 설명 사례 선택: first_positive_routing_regret. 공개 TS1 평균=35.433333, threshold=35.446367, branch=left. 감사 정답=[100, 90, 0, 90]. fit tuple 지원=False, root fit 범위 밖=False.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	1	[100, 100, 0, 90]	1	1	4/15
실제 gate	5	[100, 100, 0, 90]	1	2	17/60
두 leaf oracle	4	[100, 100, 0, 90]	1	1	4/15
전체 oracle	1	[100, 100, 0, 90]	1	1	4/15

cycle 1812 · 설명 사례 선택: first_positive_pair_gap. 공개 TS1 평균=36.191217, threshold=35.446367, branch=right. 감사 정답=[100, 73, 2, 115]. fit tuple 지원=False, root fit 범위 밖=True.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	1	[100, 100, 0, 90]	3	1	23/30
실제 gate	4	[100, 100, 0, 90]	3	1	23/30
두 leaf oracle	4	[100, 100, 0, 90]	3	1	23/30
전체 oracle	2	[20, 73, 2, 100]	2	1	31/60

first_v17_adaptive_selected_gate · g301: k=3, cap=2, λ=0

cycle 1785 · 설명 사례 선택: first_evaluation_cycle. 공개 PS1 평균=161.06345, threshold=161.03447, branch=right. 감사 정답=[100, 90, 0, 130]. fit tuple 지원=False, root fit 범위 밖=False.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	3	[3, 100, 0, 130]	2	2	1/2
실제 gate	3	[3, 100, 0, 130]	2	2	1/2
두 leaf oracle	1	[100, 100, 0, 90]	2	1	1/2
전체 oracle	1	[100, 100, 0, 90]	2	1	1/2

cycle 1788 · 설명 사례 선택: first_left_leaf. 공개 PS1 평균=161.01096, threshold=161.03447, branch=left. 감사 정답=[100, 100, 0, 130]. fit tuple 지원=False, root fit 범위 밖=False.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	3	[100, 100, 0, 130]	0	2	0
실제 gate	1	[100, 100, 0, 90]	1	1	1/4
두 leaf oracle	3	[100, 100, 0, 130]	0	2	0
전체 oracle	3	[100, 100, 0, 130]	0	2	0

cycle 1786 · 설명 사례 선택: first_right_leaf. 공개 PS1 평균=161.05212, threshold=161.03447, branch=right. 감사 정답=[100, 90, 0, 130]. fit tuple 지원=False, root fit 범위 밖=False.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	3	[3, 100, 0, 130]	2	2	1/2
실제 gate	3	[3, 100, 0, 130]	2	2	1/2
두 leaf oracle	1	[100, 100, 0, 90]	2	1	1/2
전체 oracle	1	[100, 100, 0, 90]	2	1	1/2

cycle 1789 · 설명 사례 선택: first_positive_routing_regret. 공개 PS1 평균=161.01089, threshold=161.03447, branch=left. 감사 정답=[100, 100, 0, 130]. fit tuple 지원=False, root fit 범위 밖=False.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	3	[100, 100, 0, 130]	0	2	0
실제 gate	1	[100, 100, 0, 90]	1	1	1/4
두 leaf oracle	3	[100, 100, 0, 130]	0	2	0
전체 oracle	3	[100, 100, 0, 130]	0	2	0

cycle 1787 · 설명 사례 선택: first_positive_pair_gap. 공개 PS1 평균=161.05557, threshold=161.03447, branch=right. 감사 정답=[100, 90, 0, 130]. fit tuple 지원=False, root fit 범위 밖=False.

행동	mask	예측 tuple	head 오류	quote	목적값
고정	3	[3, 100, 0, 130]	2	2	1/2
실제 gate	3	[3, 100, 0, 130]	2	2	1/2
두 leaf oracle	1	[100, 100, 0, 90]	2	1	1/2
전체 oracle	2	[20, 90, 0, 130]	1	1	1/4

19. 어떤 논문에서 무엇을 배워야 하는가

필독 축	먼저 배울 개념	ProbeFlow와 비교할 경계
GSM · Li & Oliva 2021	미관측 결과의 조건부 예측과 기대 획득 가치	현재 gate는 생성 모형을 학습하지 않음
BRiG-AFA v1	남은 horizon별 Bellman risk-to-go·동결 예측기	현재는 작은 threshold gate·비용 proxy·STOP
Hierarchical CwCF 2024	공유 원천과 계층 비용·획득 상태	현재 세 원천 실험이 일반 DAG 전체를 구현하지 않음

CwCF · Janisch et al. 2019	획득과 분류·종료를 하나의 순차 행동으로 보기	공식 초록·저자 구현 안내만 확인, native RL 미실행
DIME · Covert et al. 2023	조건부 상호정보량을 배우는 강한 greedy 대조	cross-entropy/Bayes 전제와 현재 component 목표가 다름
AFAPE · von Kleist et al. 2025	획득 정책 변경·오프라인 평가의 가정	완전 기록 재생이 물리 효과나 인과 가정을 인증하지 않음
Adaptive Submodularity	관측 이후 한계 이득 감소의 조건	XOR 보완성·공유 비용이면 자동 보장 불가
Cawley & Talbot 2010	후보 선택 전체의 평가 편향	현재 exposed 자료를 새 final로 바꾸지 않음

읽기 순서는 시스템 이해→비용·권한→고정 예측기와 oracle 분해→근시안/비근시안 비교→평가 가정이다. 한 논문의 멋진 신경망 구조만 가져오는 것으로 공정한 비교가 되지 않는다. 데이터·예측기·행동 공간·선택 정보·비용·중단·평가 접근을 한 표에 맞춘 다음 native 방법과 공통 조건 adapter를 따로 표시한다.

20. 다음에 무엇을 실제로 더 모아야 하는가

가장 중요한 다음 자료는 더 많은 같은 자료의 replay가 아니라 조건이 다양한 policy-fit과 독립적으로 예약한 장비·시간 구간이다. 먼저 어떤 고장 조합과 센서 범위를 policy가 학습할지 정하고, 실제 원천 지연·실패·공유 비용·수집 가능 시간을 측정해야 한다. 신규 전향 자료 없이 현재 결과로 일반화 성능을 보고할 수 없다.

관측한 진단	다음 설계 결정	확인할 대조
현재 두 leaf oracle도 TS1과 동률	같은 leaf의 threshold 탐색만 늘리지 않기	leaf 표현·예측기 변경을 별도 축으로 비교
k3에서 양의 routing regret	다양한 fit 조건·관측 충분성 확인	현재 gate·강한 greedy·비근시안·고정 정책
vector가 나빠지는 component 이득	진단 목적·head 가중을 사전 고정	component·vector·head별 오류 모두 보고
dev→evaluation 역전	평가 접근을 잠그고 구조적 분할 설계	독립 조건/장비·전체 실패 원장
완료 cycle 평균을 쓰는 replay	실제 시점에서 획득 가능한 관측 정의	버퍼·재측정·지연의 물리 검증

현재 연구의 의미는 실패가 없다고 포장하는 데 있지 않다. 다음 한 번의 비싼 실험에서 무엇을 바꿔야 배울 수 있는지를 분명히 하는 데 있다. v19 분해는 특히 '더 긴 계획이 필요한가'라는 질문 전에 '현재 선택지에 개선 가능성이 있는가'를 확인하게 한다.

21. 파일과 숫자를 직접 확인하는 길

PLAN.json은 새 분석이 무엇을 계산하기로 했는지, input_lock.json은 실제 읽은 코드·자료의 hash와 복사/참조 범위를, results.json은 요약을, gate_diagnostics.jsonl은 전체 gate·cohort·지원 strata를 담는다. traced_cases.json은 설명용 열 사례, verification.json은 분석기 함수를 import하지 않는 별도 산술 검산 결과다.

25,000개 캐시 행을 읽었다는 것은 25,000개 새 예측이나 독립 사이클이 생겼다는 말이 아니다. v19는 625개 기존 사이클, 한 rig, 새 parameter fit0·forward0·정책 학습0·새 선택0을 유지한다. 원천 평균 재계산, gate별 진단, 각 strata와 재생 수는 별도 ACCOUNTING에서 센다.

아래 경로는 현재 연구의 증거를 찾기 위한 주소다. 이전 실행을 덮어쓰거나 현재 계획을 사후 수정하지 않는다. 기존 검산은 읽기 전용으로 다시 실행할 수 있다. 현재 분석기는 이미 존재하는 _01 경로만을 사전에 고정해 허용하므로 그 명령으로 재생성할 수 없다. 새 실행에는 새 계획·출력 설정을 가진 별도 후보를 만들고 다시 입력·코드를 고정해야 한다.

22. 이제 자기 말로 설명할 수 있는지 점검하기

첫 설명은 60초로 해 보자. 'ProbeFlow는 관측에 따라 무엇을 더 읽을지 고른다. 같은 예측기·가격·cap으로 고정 정책과 비교한다. 현재 자료에서는 adaptive family를 넣어도 주분석은 fixed를 유지했다. v19는 그 이유를 정보의 여지, 두 leaf의 표현 한계, 라우팅 손실로 나눴다.' 이 문장마다 어떤 파일과 숫자가 근거인지 말할 수 있어야 한다.

다음 설명은 칠판에 식 없이 한다. 실제 gate를 위쪽에, 정답을 아는 두 oracle을 아래쪽에 그린다. 같은 장부에서 네 목적값 F,A,P,G를 순서대로 놓고 중간 차이를 더하면 G−F가 된다고 보여준다. 마지막에 421·1684·625·390이 각각 무엇을 세는지 구별한다. 아래 문제는 답을 열기 전에 직접 풀어 본다.

스스로 풀기와 해설

문제 1. mask5는 어떤 센서이며 왜 원천 다섯 개가 아닌가?
힌트: 101의 각 bit 위치를 펼친다.
해설: mask5는 101이므로 PS1과 TS1이다. 정수의 자릿값으로 집합을 부호화한 것이며 개수가 아니다. FS1은 포함하지 않는다. 원천 개수는 set bit 개수인 2다.

문제 2. 정책에 모든 mask의 예측과 정답을 전달해 다음 센서를 고르면 무엇이 깨지는가?
힌트: 실행과 감사의 상자를 나눈다.
해설: 정책이 아직 획득하지 않은 값의 결과와 정답을 사용하게 되어 정보 권한이 깨진다. 사후 oracle은 하한 진단에만 쓰고 fit·dev 선택·실행에는 되돌려 주지 않는다.

문제 3. TS1 평균을 보고 그 사이클 시작 시점의 실시간 진단이라고 말할 수 있는가?
힌트: 평균을 계산하는 시점을 생각한다.
해설: 현재 평균은 완료된 60초 기록에서 계산한다. 원천을 선택한 시점에 전체 구간이 이용 가능했다는 물리 검증이 없으므로 조기 경보나 실시간 추가 측정의 성과로 해석할 수 없다.

문제 4. equal quote·λ0.05에서 원천 하나를 더 읽는 목적값 증가는 얼마인가?
힌트: 비용을 세 원천 합3으로 나눈다.
해설: 같은 오류라면 한 원천의 추가 비용 항은 0.05×1/3=1/60이다. cap 안에서 추가 요청이 합법인 것과 비용 손실이 없는 것은 다르며, 원화나 지연 단위로 해석하지 않는다.

문제 5. 두 사이클의 오답 head 수가 A=(4,0), B=(1,1)이면 두 지표를 계산하라.
힌트: component 분모8, vector 분모2다.
해설: A는 component4/8와 vector1/2, B는 component2/8와 vector2/2다. B는 틀린 판단 개수를 줄이지만 두 사이클 모두 하나 이상 틀려 완전 진단 지표는 나빠진다.

문제 6. evaluation 파일을 policy lock 뒤에 열었다면 새 blind final인가?
힌트: 과거 접근 이력도 평가 계약이다.
해설: 아니다. 이번 실행의 정보 순서는 지켰지만 같은 평가 자료를 앞선 연구에서 이미 본 사실은 남는다. 새 시간 분할·새 정책·새 replay가 과거 노출을 되돌리지 않으므로 개발 진단으로 보고한다.

문제 7. adaptive_enabled의 dev 목적값이 best_fixed 이하인 것은 왜 새 일반화 정리가 아닌가?
힌트: 후보 집합 포함 관계다.
해설: adaptive_enabled가 고정 후보 전체를 포함하므로 같은 dev에서 최소값을 고르면 나쁘지 않은 것은 기계적 포함 관계다. 후보 수가 늘어 선택 편향이 생길 수 있어 evaluation의 위험 감소는 따로 검증해야 한다.

문제 8. H,E,R을 정의하고 G−F 식을 유도하라.
힌트: A와 P를 더하고 뺀다.
해설: H=F−A, E=P−A, R=G−P다. G−F=−(F−A)+(P−A)+(G−P)=−H+E+R이다. H,E,R은 같은 표본·k·가격·cap·λ에서 비음수이고 H만 마이너스로 들어간다.

문제 9. 쌍 oracle도 TS1과 같은 목적값이면 더 좋은 threshold만으로 엄격히 이길 수 있는가?
힌트: oracle은 이미 그 두 leaf의 최선이다.
해설: 현재 동결된 두 leaf와 같은 평가 기록·목적에서는 불가능하다. 실제 라우팅은 정답을 아는 쌍 oracle보다 좋을 수 없기 때문이다. 다른 leaf·예측기·자료·목표로의 일반적인 불가능성을 뜻하지는 않는다.

문제 10. 주분석 분모25260에서 −3360+2190+22를 계산하고 뜻을 설명하라.
힌트: 순변화의 부호와 각 항의 뜻을 분리한다.
해설: 합은 −1148이므로 G−F=−1148/25260=−287/6315다. PS1 대비 목적값은 좋아지지만 전체 여지3360 중 표현 gap2190과 라우팅22가 남는다. TS1 대비 개선을 뜻하지 않는다.

문제 11. k3 사례에서 −299+164+72는 무엇을 세며 vector도 좋아지는가?
힌트: component 기준과 완전 진단 기준은 다르다.
해설: 합은 −63개 component 오류이며 분모1684다. 924→625→789→861로 유도한다. 같은 gate의 vector 오류는381→391이므로 완전 진단은 오히려 나빠진다. 서로 다른 분모와 목표를 혼합하지 않는다.

문제 12. fit에서 본 tuple이 evaluation30/421에 있다는 말은 tuple 종류가30개라는 뜻인가?
힌트: 종류와 행 수를 나눈다.
해설: 아니다. fit에는 한 tuple 종류만 있고 그 tuple을 가진 평가 사이클이30개다. 평가에는 모두38종류의 tuple이 있다. 나머지391개 사이클은 fit에서 보지 못한 tuple을 갖지만 종류391개라는 뜻은 아니다.

문제 13. 390 gate와 수십만 진단 행이 통계 표본을 늘리는가?
힌트: 같은625개 원자료를 반복 사용한다.
해설: 늘리지 않는다. 설정·cohort·strata별 계산은 같은 장비와 사이클을 반복해서 사용한 산술 단위다. 새 독립 장비·고장·시간 단위를 얻지 않았으므로 이 수를 독립 표본 분모에 넣으면 안 된다.

문제 14. 정보 이득 greedy가 현재 component 목적의 최적 정책이라는 보장을 바로 가져와도 되는가?
힌트: 원문의 loss와 predictor 가정을 확인한다.
해설: 안 된다. 조건부 상호정보량과 Bayes·cross-entropy 관계는 특정 분포·예측기·목표의 조건을 요구한다. 현재 동결 kNN과 네 head의 component 손실·비용·STOP 계약은 별도로 맞춰 비교해야 한다.

문제 15. 현재 replay가 원천 비용 절감을 실제 장비에서 입증하지 못한 이유 두 가지는?
힌트: 이미 모두 저장한 자료와 proxy quote다.
해설: 원자료는 이미 전체 수집되어 있고 quote는 표본 수 또는 equal 단위의 proxy다. 실제 전력·지연·실패를 측정하지 않았으며 과거 같은60초를 다시 읽는 물리 가능성도 검증하지 않았다.

문제 16. 새 자료가 한 번만 가능하다면 무엇부터 고정할 것인가?
힌트: 모델 크기보다 자료·권한·목표·대조 계약을 먼저 쓴다.
해설: 다양한 policy-fit 조건과 독립 예약 장비·시간 구간, 획득 가능한 관측 시점, 실제 비용, 목표 손실, fixed/greedy/비근시안 대조, 실패·미도달·보류 분모를 먼저 고정한다. oracle 진단으로 새 평가 정책을 사후 선택하지 않는다.

원문과 확인 범위

- UCI Hydraulic 원자료 설명: https://archive.ics.uci.edu/dataset/447/condition+monitoring+of+hydraulic+systems

센서·사이클·목표의 공식 데이터 설명. 기존 원고의 공식 자료 검토와 로컬 파일 차원을 대조했다. 새 장비나 실제 비용을 확보하지 않았다.

- GSM · ICML 2021: https://proceedings.mlr.press/v139/li21p.html

공식 논문 페이지·기존 정독실의 선택 절. 원문의 획득 가치 개념과 현재 단순 gate의 차이만 연결한다. native 구현 미실행.

- BRiG-AFA · 고정 v1: https://arxiv.org/html/2608.02305v1

§3–4.2, 동결 예측기·horizon·위험 회귀. 기존 정독실과 이번 공식 본문 확인을 연결했다. 새로운 회귀기나 원문 수치 재현은 없다.

- Hierarchical CwCF 2024: https://link.springer.com/article/10.1007/s10994-024-06565-4

기존 정독실의 비용·계층 구조 선택 절. 실제 일반 DAG나 원문 A2C를 재현하지 않았다.

- CwCF · AAAI 2019: https://ojs.aaai.org/index.php/AAAI/article/view/4287

공식 서지·초록·저자 구현 안내. 획득과 분류/종료 행동, 비용-오류 목적의 저자 보고만 짧게 연결한다. 전체 본문·코드 감사는 하지 않았다.

- DIME · ICML 2023: https://proceedings.mlr.press/v202/covert23a.html

공식 서지·초록 및 웹 본문의 §3.2, §4.1. greedy CMI와 학습 대리 결정, cross-entropy·Bayes 조건을 선택 검토했다. 전체 증명·실험 미재현.

- AFAPE · JMLR 2025: https://www.jmlr.org/papers/v26/23-1635.html

공식 서지·초록의 획득 분포 변화·NDE/NUC. 전체84쪽과 모든 식을 검토한 것으로 표시하지 않는다. 현재 유압 replay의 인과 가정을 인증하지 않는다.

- Adaptive Submodularity · 정정 v5: https://arxiv.org/html/1003.3967v5

기존 정독실의 정의·가정·XOR 반례. 새 데이터에서 정리 조건이 성립한다는 보장을 주장하지 않는다.

- Cawley & Talbot 2010: https://jmlr.org/papers/v11/cawley10a.html

기존 정독실의 선택·평가 구분. 새 독립 평가를 얻지 않은 현재 한계와 연결한다.

v19 전체 진단·기록 추적 부록

ProbeFlow v19 해설: 무엇이 아직 가능했고, 어디에서 그 가능성을 잃었는가

이번 후속 연구의 질문은 "복잡한 정책이 좋아 보이는가"가 아니다. 현재 고정 예측기와 비용 계약에서 개선 여지가 있었는지, 지금 허용한 두 종료 선택이 그 여지를 포함하는지, 관측값을 본 정책이 그 선택을 잘 골랐는지를 따로 묻는다. 세 질문을 한 성능 숫자로 뭉치면 개선할 부위를 찾기 어렵다.

우선 자료의 성격을 고정하자. 한 장비의 기존 기록이며, 정책 fit 102사이클·정책 dev 102사이클·이미 여러 판본에서 본 평가 421사이클을 쓴다. 네 목표는 cooler·valve·pump·accumulator다.

component error는 네 판단 중 틀린 수의 합을 4N으로 나누고, vector error는 네 판단 중 하나라도 틀린 사이클을 N으로 나눈다. 평균 component 오류가 줄어도 완전한 진단이 늘었다고 말할 수 없는 이유다.

1. 실제로 무엇을 다시 계산했는가

기존 v17에서 학습한 gate를 그대로 가져왔다. 각 gate는 먼저 센서 하나의 완성된 사이클 평균을 관측하고 임계값과 비교한 뒤, 고정된 두 leaf 중 하나로 끝난다. mask의 비트는 PS1=1, FS1=2, TS1=4다. 예를 들어 mask 5는 PS1+TS1, mask 3은 PS1+FS1이다. 이는 한 사이클의 raw trace를 순서대로 조금씩 보는 온라인 제어 실험이 아니다. 이미 끝난 60초 기록에서 허용된 평균을 공개하는 재생이다.

새로 한 일은 고정된 모든 mask의 기존 예측을 실제 정답과 대조하고, 동일한 계약에서 다음 네 값을 계산한 것이다.

1. locked fixed: v17 dev 절차가 이미 선택한 같은 k의 고정 subset. 2. 실제 gate: v17에서 이미 고정한 root·threshold·두 leaf로 결정한 종료 mask. 3. pair oracle: 실제 정답까지 알고 그 두 leaf 중 매 사이클 더 좋은 것을 고르는 가상의 하한. 4. all-mask oracle: 같은 cap에 허용되는 모든 mask 중 매 사이클 더 좋은 것을 고르는 더 낙관적인 하한.

3과 4는 배포 가능한 정책이 아니다. 정답과 미획득 source를 썼을 때의 예측을 미리 본다. 따라서 이들의 점수가 좋다는 사실은 그만큼 현실에서 얻을 수 있다는 뜻이 아니다. 다만 이런 특권을 주어도 좋아질 수 없다면, 같은 후보집합의 합법적 정책이 엄격히 더 좋아지기는 불가능하다는 조건부 진단은 할 수 있다. 이 경계가 oracle을 사용하는 이유다.

2. 세 항의 식을 이해하기

각 사이클의 점수는 아래와 같다. component 오류는 0·1·2·3·4개 중 하나이고, quote는 그 종료 mask에 선언한 가격이다.

u_i(mask) = 해당 사이클의 틀린 head 수 / 4 + λ × quote(mask) / 전체 source 가격의 합
같은 사이클들의 평균을 U라 쓰면,
H = U(locked fixed) − U(all-mask oracle) E = U(pair oracle) − U(all-mask oracle) R = U(actual gate) − U(pair oracle)
U(actual gate) − U(locked fixed) = −H + E + R
H는 특권 정보를 가진 모든 mask oracle에 비해 기준 고정 선택에 남은 여지다. E는 그 넓은 oracle에 비해 현재 두 leaf만 허용했을 때 잃는 양이다. root를 반드시 읽는 제한, 다른 source 조합을 못 쓰는 제한, 해당 leaf의 가격까지 함께 포함한다. 이를 신경망의 순수한 표현력 효과라고 부르면 안 된다. R은 같은 두 leaf 안에서 실제 gate가 특권 oracle에 비해 더 낸 손실이다. 이 역시 관측 가능한 root 정보만으로 반드시 줄일 수 있는 학습 오류라고 단정할 수 없다.
세 값 자체는 모두 0 이상이다. 그러나 H는 최종 차이에서 음수로 들어간다. 실제 gate가 좋은지는 이용 가능한 여지 H가 후보 제한 E와 라우팅 차이 R의 합보다 큰지에 달려 있다. 이 식은 최적화 알고리즘의 신기한 새 정리가 아니라 같은 네 숫자를 빼고 더한 정확한 회계 항등식이다. 실제 자료에서 각 항을 고정하고 실패를 구별할 수 있게 실행한 것이 이번 진전이다.

3. primary: 적응의 성공처럼 보인 78개 오류 감소를 다시 읽기

사전 지정 primary는 k1·equal source quote·cap 2·λ=1/20다. source 세 개의 가격은 각각 1이므로 비용 정규화 분모는 3이다. v17 dev의 locked fixed는 PS1(mask 1)이다. forced gate g129는 다음 규칙이다.
먼저 TS1 평균을 읽는다. TS1 평균 ≤ 35.44636666666667 이면 PS1도 읽고 mask 5로 종료한다. 그보다 크면 TS1만 쓴 mask 4로 종료한다. 정확한 경계: 0x1.1b9228afdadcfp+5
평가 421사이클에서 고정 PS1은 component 오류 876/1684, gate는 798/1684다. 하지만 vector 오류는 둘 다 391/421이고, quote 합은 421→443이다. 작은 수치로 바꾸지 않고 정확한 목적값을 놓으면 다음과 같다.

객체	정확한 평균 목적값
locked fixed PS1	13561/25260
실제 gate	12413/25260
두 leaf oracle	12391/25260
모든 feasible mask oracle	10201/25260

따라서 H=56/421, E=73/842, R=11/12630이고, −H+E+R=−287/6315다. PS1 대비 gate의 목적값 개선은 약 0.04545다. 이 값만 보여 주면 적응 정책의 승리처럼 보일 수 있다.
더 강한 통제는 이미 있던 TS1/k1이다. 그것은 798 component 오류와 quote 421을 가지므로 목적값이 정확히 12391/25260이다. 이 값이 pair oracle의 하한과 같다. 그러므로 현재 두 leaf 4/5 사이에서 어떤 방식으로 라우팅하더라도, 이 고정 예측기·가격·평가 구간에서는 TS1 상수를 엄격히 이길 수 없다. 새 임계값을 탐색하는 것만으로 이 두 선택 안의 문제가 해결될 것이라는 주장은 이 하한과 맞지 않는다. 다른 leaf·다른 예측기·다른 자료는 별개의 후속 질문이다.
실제 gate가 pair oracle보다 낸 11/12630 = 22/25260은 이 설정에서는 같은 예측 오류에 덧붙인 22 quote의 벌점과 같다. 이것은 실제 산업적 검사비를 측정한 금액이 아니라 논리 장부다. PS1보다 좋아 보인 효과를 "관측값에 맞게 분기해서 얻은 순이득"이라고 바로 해석할 수 없는 이유가 여기에 있다.

4. primary를 사이클 단위로 따라가기

정답 tuple과 예측 tuple의 차이는 head별 오류 수로 센다. 아래 사례는 사전 선언한 "가장 이른 각 종류의 사례" 규칙으로 뽑은 사후 설명용 기록이다. 이 다섯 사례의 평균으로 성능을 추정하지 않는다. 정확한 float.hex·전체 예측·가격은 traced_cases.json에 있다.

cycle	TS1 관측 평균	임계 비교	gate mask	고정/gate/pair/all 오류 수	고정/gate/pair/all quote
1785	36.2273	초과→right	4	2 / 2 / 2 / 2	1 / 1 / 1 / 1
2184	35.442933333333336	이하→left	5	1 / 1 / 1 / 1	1 / 2 / 1 / 1
1786	36.21248333333334	초과→right	4	2 / 2 / 2 / 2	1 / 1 / 1 / 1

2185	35.43333333333333	이하→left	5	1 / 1 / 1 / 1	1 / 2 / 1 / 1
1812	36.19121666666667	초과→right	4	3 / 3 / 3 / 2	1 / 1 / 1 / 1

cycle 2184의 정답은 (100,90,0,90)이고 네 비교의 예측은 모두 (100,100,0,90)이다. valve 한 판단만 틀린다. 고정 PS1·TS1 pair oracle은 1/4+1/60=4/15, gate는 PS1까지 추가로 읽어 1/4+2/60=17/60을 낸다. 예측은 그대로이고 quote만 늘어난 경우다.

cycle 1812의 정답은 (100,73,2,115)다. gate의 두 leaf로는 예측 (100,100,0,90)에서 세 head가 틀린다. 모든 mask를 미리 보는 oracle은 FS1(mask 2)의 (20,73,2,100)을 골라 두 head만 틀린다. 여기서는 임계값을 잘못 분기한 것이 아니라 현재 두 leaf에 FS1 단독이 없다는 제약이 드러난다. 그렇다고 실제 정책이 정답을 모르는 상태에서 FS1을 고를 수 있었다는 증거가 생긴 것은 아니다.

5. secondary: 더 나은 평균 진단과 더 나쁜 완전 진단

두 번째 사례 g301은 v17에서 adaptive_enabled가 gate를 고른 것들 중 사전 정한 정렬로 첫 번째다. k3·equal cap 2·λ=0이다. 이미 고정된 규칙은 PS1 평균을 161.03447166666666과 비교해 이하이면 mask 1, 초과이면 mask 3을 선택한다. 정확한 경계는 0x1.4211a64531f1ap+7이다.

λ가 0이므로 여기의 세 항은 component 오류 개수로도 바로 읽을 수 있다.

locked fixed 오류 = 924 all-mask oracle 오류 = 625 pair oracle 오류 = 789 실제 gate 오류 = 861

−(924−625) + (789−625) + (861−789) = −299 + 164 + 72 = −63

모두 같은 1684 head 판단으로 나누면 H=299/1684, E=41/421, R=18/421, 실제 차이=−63/1684다. gate는 63개 component 오류를 줄인다. 그러나 완전 진단 오류는 381→391로 10사이클 늘고, quote 합은 842→518로 줄어든다. 좋은 평균·더 싼 정보·나빠진 전체 진단이 함께 존재한다. 사용자나 논문이 무엇을 목표로 하는지에 따라 이 교환의 의미가 다르므로 하나의 "성능 향상"으로 축약하지 않는다.

cycle	PS1 평균	gate mask	정답	gate 예측	fixed 오류→gate 오류
1785	161.06345166666665	3	(100,90,0,130)	(3,100,0,130)	2→2
1788	161.01096166666667	1	(100,100,0,130)	(100,100,0,90)	0→1
1786	161.05212166666666	3	(100,90,0,130)	(3,100,0,130)	2→2
1789	161.01089166666665	1	(100,100,0,130)	(100,100,0,90)	0→1
1787	161.05556666666666	3	(100,90,0,130)	(3,100,0,130)	2→2

cycle 1788에서 gate는 PS1만 읽고 멈춰 quote 1에 head 하나를 틀린다. 고정 PS1+FS1은 quote 2로 네 head를 맞힌다. pair oracle는 정답을 미리 보아 고정 mask 3을 택한다. 이것은 관측 PS1 값만으로 이 손해를 구별할 수 있었다는 학습 가능성의 증명이 아니라, 실제 gate와 특권 하한 사이의 1/4 위험 차이를 추적하는 기록이다.

6. 지원 붕괴와 바깥 범위를 어떻게 읽어야 하는가

fit 102개 목표 tuple은 하나뿐이다. 그 tuple은 dev의 2/102, 평가의 30/421에서만 나타난다. g129의 root TS1 평균은 dev 100/102, 평가 392/421에서 fit 최솟값·최댓값 바깥이다. g301의 PS1 평균은 dev 99/102, 평가 382/421에서 바깥이다. 이는 학습된 문턱을 다른 구간에 적용하고 있음을 드러내는 유한 진단이지, 밖의 모든 예측이 틀린다는 정리는 아니다.

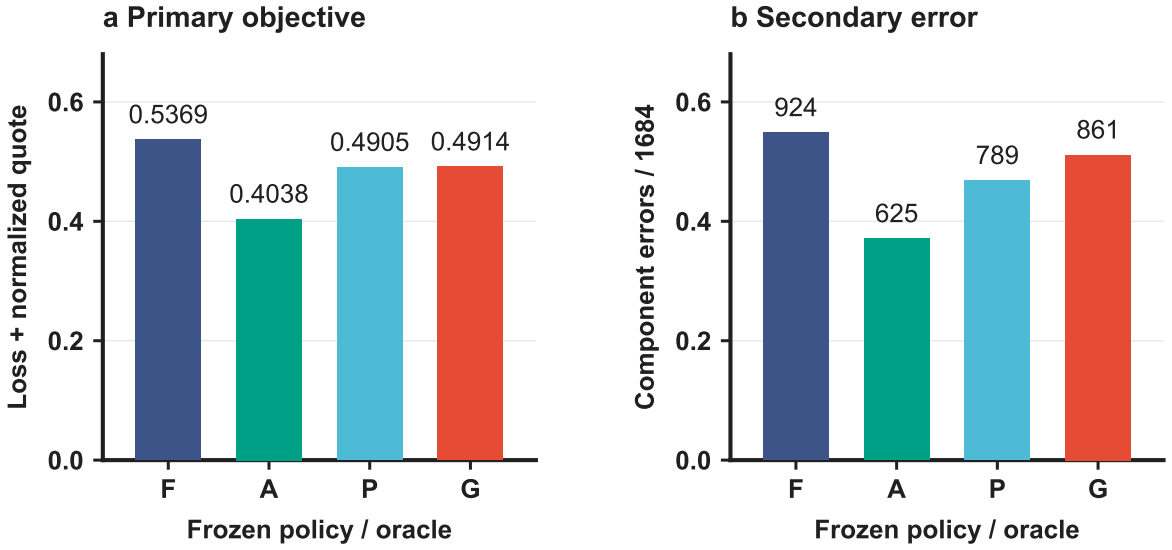
실제로 primary의 fit-seen 30사이클에서는 모든 mask oracle에 대한 headroom이 0인데 gate가 추가 quote를 내어 차이가 19/1800으로 나빠진다. fit-unseen 391사이클에서는 PS1 대비 차이가 −389/7820으로 좋아진다. 이 결과로 정답 tuple을 실행 중 구별하는 새 정책을 만들지 않았다. seen/unseen은 정답을 본 뒤 붙인 진단용 strata다.

핏 구간에서 동일 목표만 봤다고 해서 모든 관측 기반 행동 가치 학습이 수학적으로 불가능하다고 말할 수도 없다. 같은 목표 안에서도 예측기의 실수와 관측값의 관계를 배울 수 있다. 다만 보지 않은 목표 조건에서 그 관계가 유지되는지는 자료가 알려 주지 않는다. 다음 실험에는 더 다양한 policy-fit 조건과 미리 예약한 독립 장비·구간이 필요하다. 이 작업에서 그 자료를 확보하거나 새 평가를 진행한 것은 아니다.

7. 무엇이 검증됐고 무엇이 남았는가

독립 검산기는 분석 함수를 import하지 않고 원시 source hash·625×3 평균·모든 cached prediction·1,170행과 4,680 stratum을 다시 계산했다. 일부러 두 분해 값을 바꾼 fixture를 거절했고, 두 사례의 10사이클 예측과 관측을 확인했다. 이는 산술과 입력 연결에 대한 검증이다. 원문 BRiG-AFA나 GSMRL을 재현했다는 뜻도, 실제 센서 요청의 비용·안전·시점이 같다는 뜻도 아니다.

다음 결정을 세 문장으로 설명할 수 있어야 한다. "이 두 leaf를 계속 쓰며 임계값만 더 학습할 이유가 있는가?", "더 넓은 후보에 잠재 여지가 있더라도 현재 관측으로 그것을 알아볼 수 있는가?", "평균 오류를 줄이면서 늘어난 완전 진단 실패를 어떤 목표에서 허용하는가?" 이번 연구는 이 질문에 필요한 장부를 실제 자료와 함께 만들었다. 하나의 좋은 oracle 숫자를 얻었다는 이유로 답을 자동 결정하지는 않는다.



F: fixed A: all-mask oracle P: pair oracle G: gate

Saved 421-cycle evaluation; A/P use labels retrospectively.

그림 1: 이번 후속 분석

이전 연구의 원리·실험·문헌 해설 · v18 보존본

ProbeFlow v17 해설: 관측에 따라 센서를 더 사면 정말 나아지는가

이번에는 실제로 달라진 계산이 무엇인가

v16까지의 핵심 비교는 입력값에 따라 획득 순서가 바뀌지 않는 고정 정책이었다. v17은 같은 cycle의 첫 센서 평균을 실제로 공개한 뒤, 그 관측값이 threshold 이하인지에 따라 멈추거나 다른 센서를 하나 더 요청하는 정책을 실행한다. 따라서 관측 의존의 적응형 정책 코드는 새로 실행됐다. 다만 이미 기록된 60초 cycle의 데이터를 가려 두었다가 공개하는 오프라인 replay이며, 장비를 켜서 센서를 추가 측정한 물리 실험은 아니다. 완료된 cycle의 전체 평균을 본 뒤 이미 지나간 같은 시간대의 다른 센서를 뒤늦게 측정할 수 있는지는 별개다. 버퍼·재측정·지연 조건을 검증하지 않았으므로 실제 온라인 센서 제어나 조기 경보의 구현으로 해석할 수 없다.

말의 범위를 구별해야 한다. 정책의 branch action은 이번에 새로 학습했다. 단말 예측기는 다시 학습하지 않았고, 기존 v15의 k1/k3 예측 캐시를 선택된 terminal mask에 맞춰 조회한다. 새 gate 학습을 해 놓고 "새 parameter fit은 전부 0"이라고 쓰면 틀리다. 반대로 캐시를 재사용했다는 이유로 새

적응형 실행도 0이라고 쓰면 관측 권한을 지킨 정책 replay가 사라진다. 학습·정책 선택·실행·독립 표본을 각각 집계한다.

predictor train, policy fit, policy dev, exposed evaluation

단말 예측기의 학습 bank는 기존 1..1540 cycle이며 그대로 동결한다. 과거 selection 204개 중 앞 102개인 1561..1662는 gate의 threshold와 branch action을 만드는 policy fit이다. 뒤 102개인 1663..1764는 이미 만들어진 후보 중 정책을 고르는 policy dev다. 1785..2205의 421개는 정책을 고정한 후 채점하지만, 앞선 연구에서 이미 노출된 evaluation이다. 새 시간 분할을 만들었다고 독립 final이 되지 않는다.

코드는 현재 실행에서 evaluation의 평균·정답·예측 records를 읽기 전에 policy_locks.json을 기록한다. profile 파일 전체를 snapshot/hash하는 일과 evaluation 정답을 정책 학습에 사용하는 일은 다르다. 그래도 과거 노출 사실을 되돌릴 수는 없으므로 이를 새로운 blind test로 부르지 않는다. 한 장비의 625개 기존 cycle이 반복해서 쓰이고, 수십만 trajectory가 새로운 실험 표본이 되는 것도 아니다.

정책이 정확히 볼 수 있는 정보

정책 결정 함수의 인자는 policy, observed, acquired뿐이다. policy는 학습이 끝난 규칙, observed는 이미 요금을 내고 획득한 source의 평균 mapping, acquired는 획득 mask다. cycle ID, target, 미도착 source 값, terminal prediction bank를 전달하지 않는다. 처음에는 observed가 비어 있으므로 학습된 root source만 요청한다. root의 실제 평균을 받은 뒤에만 왼쪽·오른쪽 leaf를 선택한다.

leaf는 root만 쓰고 STOP하거나, root와 다른 source 하나를 함께 쓰는 mask다. 두 leaf의 mask는 다르도록 제한한다. 이 gate는 최대 두 source를 요청한다. source마다 mean·standard deviation·slope를 쓴 기존 단말 예측기와 구별해서, 정책의 분기에는 mean 하나만 허용한다. source 전체를 샀다는 quote가 해당 source의 단말 특징 이용을 포함하며, 아직 안 산 source의 특징을 단말 예측에 쓰는 것은 허용되지 않는다.

이것은 조사 가능한 Python API 경계다. 별도 프로세스·OS 보안 격리나 악성 코드를 막는 sandbox라고 쓰지 않는다. 기존 raw 파일은 모두 존재하지만, 신뢰한 정책 함수에 무엇을 전달했는지와 실제 request trace를 검토할 수 있게 만든 실험이다.

작은 gate를 학습하는 방법

각 source의 policy-fit 평균 102개를 정렬하고 1/4, 1/2, 3/4 위치의 값을 threshold 후보로 쓴다. 정확한 index는 floor((102-1)q)다. 비교 값은 float.hex로 기록하므로 소수점 문자열을 짧게 출력하면서 branch 경계가 바뀌는 일을 피한다. 양쪽 fit 표본은 각각 최소 16개여야 한다.

각 root·threshold에 대해 예산 안에 들어오는 두 개의 서로 다른 leaf mask를 모두 비교한다. 목적은 fit의 네 head 평균 component 오류에 λ 곱하기 평균 source quote/전체 source quote 합을 더한 값이다. 동률이면 낮은 평균 quote, 낮은 왼쪽 mask, 낮은 오른쪽 mask를 고른다. branch action은 fit 정답으로만 학습한다. dev·evaluation의 정답으로 leaf를 고치지 않는다.

이 정책은 Bellman network, deep RL 또는 native BRiG-AFA가 아니다. 의미 있는 최소 관측 의존 정책을 실행해, 기존 predictor와 현재 자료에서 acquisition을 복잡하게 만드는 것이 어떤 종류의 이익·실패를 만드는지 검사하는 bounded baseline이다. 특정 학회 수준의 novelty나 모든 AFA보다 낫다는 주장은 하지 않는다.

고정 정책을 일부러 약하게 만들지 않는다

best_fixed는 예산이 허용하는 여덟 source mask를 전부 비교한다. empty mask와, 허용된다면 세 source를 모두 쓰는 full mask도 들어 있다. gate는 최대 두 source만 쓰므로 모든 조건에서 gate가 더 넓은 모델 class인 것은 아니다. adaptive_enabled는 이 모든 고정 후보와 fit에서 학습한 gate 후보를 합친 집합이다. 이 합집합의 dev 최저 목적값이 best_fixed보다 나쁘지 않은 것은 후보 포함 관계에서 나오는 성질이지 일반화 발견이 아니다.

gate_only는 서로 다른 leaf를 가진 gate 중 dev 목적값이 가장 좋은 것을 고른다. gate를 살 수 없는 예산에서는 best_fixed로 돌아오며 gate_available=false를 남긴다. gate_only는 "적응형 정책을 실제

로 쓰면 어떤 일이 생기는가"를 보여주는 진단이다. adaptive_enabled가 고정 정책을 택했는데 gate_only의 evaluation 숫자를 대신 성공 결과로 내세우면 선택 계약을 바꾼 것이다.
세 family 모두 동일한 predictor k, 같은 per-cycle hard cap, 같은 λ, 같은 policy-dev objective를 쓴다. 하지만 이 조건이 실제 평균 지출까지 같다는 뜻은 아니다. fit/dev/evaluation별 terminal-mask 빈도와 quote 합·평균을 함께 보고한다. raw 가격표와 equal 가격표는 서로 다른 proxy 단위이고, 실제 돈·시간·에너지나 offline 전체 데이터 수집비가 아니다.

사전에 고정한 주분석은 적응형 정책을 채택하지 않는다

주분석은 equal quote의 cap 2, λ=0.05, k1이다. best_fixed는 PS1 하나를 선택한다. adaptive_enabled도 같은 PS1을 유지한다. dev component 오류와 objective가 같은 gate보다 단순한 fixed를 먼저 두는 동률 규칙까지 계획에 고정했다. evaluation component 오류는 두 정책 모두 876/1684이고 vector 오류는 391/421이다. 이 조건에서는 adaptive family를 추가해도 선택된 행동과 예측은 개선되지 않았다.
gate_only는 TS1 평균을 먼저 보고, threshold 이하에서는 PS1을 더 사고 그 외에는 STOP하는 정책이다. fit에서는 26개가 TS1+PS1, 76개가 TS1만 쓴다. dev 102개에서는 모두 TS1만 쓰고 추가 source를 요청하는 branch가 한 번도 검증되지 않는다. evaluation에서는 22개가 TS1+PS1, 399개가 TS1만 쓴다. complete-cycle mean의 분포가 달라지면 평균 지출도 달라질 수 있음을 보여준다.
이 강제 gate의 evaluation 오류는 798/1684로 PS1의 876보다 작다. 그러나 이미 있던 TS1/k1 고정 정책도 798/1684이고 vector 오류 역시 둘 다 391/421이다. gate는 quote를 421에서 443으로 늘려 22개를 더 지출하면서 단말 오류를 줄이지 못한다. 따라서 강한 같은-source control을 남기면 이 결과는 적응형 가치의 입증이 아니라 불필요한 추가 요청의 반례가 된다. 과거 TS1 control은 selection 접근이 현재 selector와 다르므로 역사적 진단으로 명시하며, evaluation을 보고 새 승자로 채택하지 않는다.

k3 민감도는 지표 간 충돌과 전이 실패를 보여준다

전체 90개 k·가격표·cap·λ 조건 중 adaptive_enabled가 gate를 고른 것은 6개다. 그중 component 오류가 낮은 것은 4개, 높은 것은 2개이며 나머지 84개는 동률이다. 이 수치는 같은 gate를 cap 2와 3, 여러 λ에서 반복한 결과다. 여섯 번의 독립적인 실험 성공·실패로 해석하지 않는다.
k3, equal cap 2에서 λ=0 또는 0.01이면 고정 비교는 PS1+FS1이다. 선택된 gate는 PS1을 관측하고 평균이 threshold 이하이면 PS1만으로 멈추며, 높으면 FS1을 더 요청한다. evaluation component 오류는 fixed 924에서 gate 861로 63개 감소하고 quote 합은 842에서 518로 준다. 하지만 vector 오류는 381에서 391로 10개 늘어난다. component를 평균내 얻은 이익이 네 조건을 전부 맞추는 진단의 이익을 보장하지 않는다. 서로 다른 metric의 분모도 1684와 421로 다르다.
같은 k3 gate라도 λ=0.05에서는 비교할 fixed가 PS1 하나로 바뀐다. dev에서는 오류가 248에서 245로 세 component 줄어 gate가 채택되지만, evaluation에서는 799에서 861로 62개 늘어난다. quote 합도 421에서 518로 97 증가한다. vector 오류는 391로 같다. 같은 λ와 cap을 지켰다는 사실만으로 평가에서 더 싸고 정확한 정책이 되는 것은 아니다. cap 3에서도 이와 같은 정책과 결과가 반복된다.

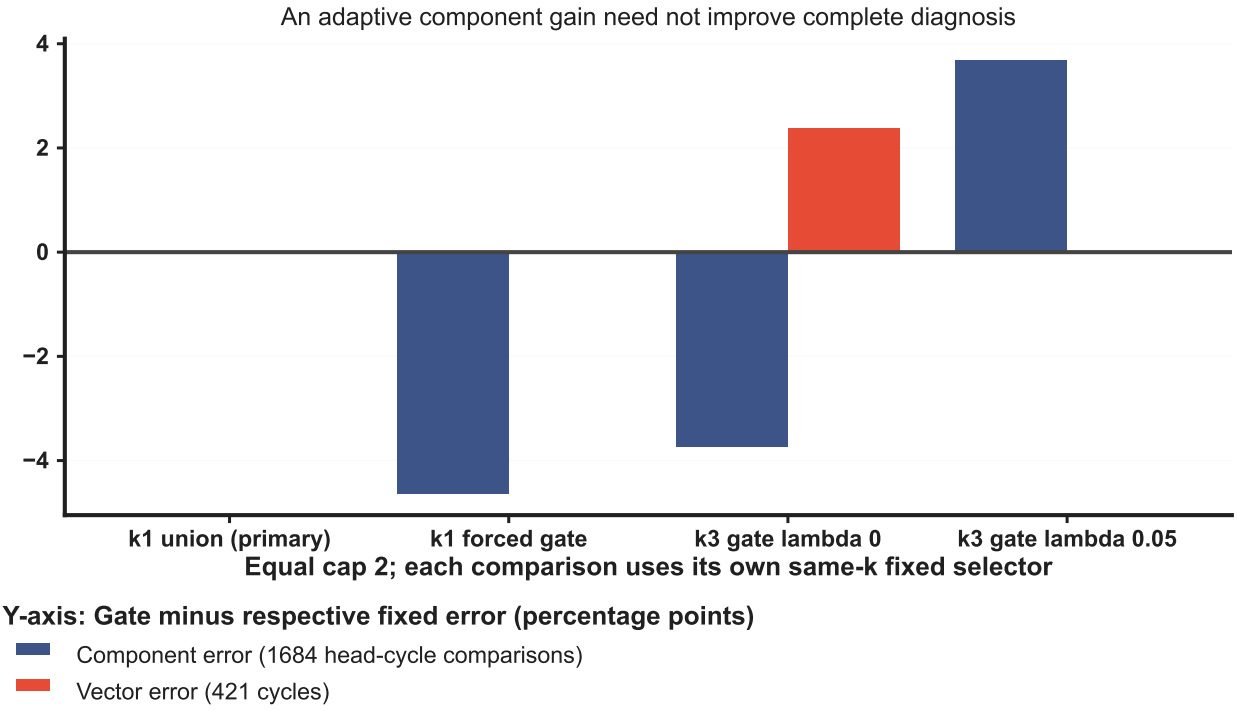


그림 2: 관측 gate의 오류·비용 대조

그림 8. equal cap 2에서 k1 주분석 union, k1 강제 gate, k3 λ=0 gate, k3 λ=0.05 gate를 각각 같은 k의 해당 고정 정책과 대조한 오류율 차이다. component는 1,684개 head-cycle 비교, vector는 421개 cycle을 분모로 한 percentage point이며 비용 막대는 아니다. 주분석 union의 차이는 0이다. 강제 k1 gate의 component 오류 798/1684는 비교 PS1의 876/1684보다 작지만 기존 TS1 control의 798/1684와 같고, quote는 TS1의 421보다 22 큰 443이다. k3 λ=0에서는 component 오류 감소와 vector 오류 10개 증가가 공존하고, λ=0.05에서는 component 오류가 62개 늘어난다. 모두 이미 노출된 한 장비의 같은 421개 evaluation cycle에 대한 오프라인 대조이며, 실제 센서 비용 절감이나 독립 전이 우월성을 뜻하지 않는다.

앞 102개의 정답이 모두 같다는 발견을 숨기지 않는다

계획을 바꾸지 않고 결과를 해석하는 후속 진단에서, policy fit 102개의 정답 tuple이 모두 (100, 100, 0, 90)임을 확인했다. dev에는 그 tuple이 2/102개, evaluation에는 30/421개만 남는다. 나머지는 fit에서 한 번도 보지 못한 joint condition이다. 이 통계는 동결된 profile에서 계산한 post-hoc 진단이며, 이미 고정된 policy를 바꾸거나 oracle 정보를 제공하는 데 쓰지 않았다.

따라서 서로 다른 fault 조건에 대한 acquisition utility를 fit 구간이 충분히 알려줬다고 할 수 없다. fit에서 오류가 없는 행동끼리 gate의 두 leaf를 다르게 강제하면 비용과 동률 규칙이 branch를 결정할 수 있다. "390개 gate를 학습했다"는 실행 사실과 "다양한 조건의 정책을 식별했다"는 과학적 주장은 전혀 다르다. 분할을 사후로 섞어 더 좋은 숫자를 만든 뒤 처음부터 정한 실험처럼 보고하지 않는다. 현재 실험의 결과는 관측 권한을 지킨 코드가 존재한다는 것, 강한 fixed control을 생략하면 잘못된 이익을 읽을 수 있다는 것, 그리고 이 자료 분할로는 robust adaptation을 확인할 수 없다는 것이다. 다음 실험은 policy-fit 조건 지원과 독립 평가 장비·조건을 먼저 확보한 뒤 split·목표·threshold class·λ·실제 비용 계약을 미리 고정해야 한다.

숫자를 세는 단위

새로 학습한 gate model은 390개, leaf action은 780개, fit에서 정한 threshold 값은 9개다. dev 후보 채점은 790개이고 최종 family 선택 record는 270개다. 이들은 같은 자료를 쓰는 계산 단위이지 독립 표본이 아니다. 선택 정책 replay trajectory는 168,750개이고 그중 nonconstant gate를 실행한 것은 35,000개다. 한 trajectory는 source 요청이 0~3개인 cycle 1회의 경로다. 따라서 trajectory 수를 센서

요청 수라고 쓰지 않는다. raw trace의 requests 길이를 합친 실제 source 요청은 130,459건이며 ACCOUNTING.json에 별도로 남겼다.
위 selected-policy 수에는 후보를 dev에서 채점한 80,580개 trajectory와 역사적 fixed control 진단 842개는 들어 있지 않다. 숨은 값에 ±1e90를 넣어 행동이 바뀌지 않는지 확인한 1,560개의 실행은 합성 권한 경계 fixture이며 실제 cycle 표본이 아니다. 새로운 predictor parameter fit·forward call, 새 장비, 새 데이터 다운로드, 모델 API, 물리 센서 실험은 0이다.

검산이 입증하는 범위

verify.py는 analyze.py를 import하지 않는다. 원본 raw 평균과 동결 예측·정답으로 threshold, branch action, dev 후보, 선택 정책, 세 cohort의 head/vector 오류와 quote, 요청 순서·실제 공개 값·terminal mask를 다시 계산한다. 25,000개 기존 예측과 168,750개 선택 경로를 확인한다. 실제 controller에 미도착 source 값을 바꾸어 넣는 경계 fixture도 별도 기록한다.
이 대체 구현은 같은 저자가 작성한 로컬 산술·경로 검산이다. 독립 외부 연구자의 재현이나 과학적 타당성 심사로 표시하지 않는다. root 통합 lane의 비저자 검토와 PDF 확인은 별도이며, 그것도 독립 장비에서의 모델 성능을 대신하지 않는다.

핵심 문헌과 구현 차이

BRiG-AFA v1의 공식 HTML에서는 관측값·mask·후보·남은 budget만을 쓰는 배포 정보 계약, 고정 predictor를 공유하는 비교, 그리고 단위비용·고정 horizon의 제한을 확인했다. 이번 gate는 그 알고리즘을 재현한 것이 아니다. Bellman risk-to-go network를 학습하지 않고, source 비용 proxy와 STOP을 가진 작은 로컬 정책 class를 쓴다. native AFA와 성능표를 합치지 않는다.
Janisch·Pevný·Lisý의 공식 publisher HTML에서는 cost-weighted objective와 budget constraint의 차이 및 validation 선택 범위를 재확인했다. hierarchical A2C, 해당 논문의 데이터와 실제 측정 비용은 여기서 재현하지 않았다. 두 문헌은 방법 비교의 공정 조건을 정하는 출처이며, 저자들의 외부 수치를 이번 one-rig 실측으로 옮기지 않는다.

선택한 연구 관점과 남은 일

이번 차수는 강한 baseline·ablation을 점검하는 Kaiming He 관점과 분포·일반화 범위를 점검하는 Yoshua Bengio 관점이 필요하다. 이는 그 연구자들의 공개 연구 방법론을 적용한 Native Codex 검토 요청이며 직접 교수 자문이 아니다. 새로운 인물을 더 부르는 것보다 primary fixed 유지, TS1 반례, k3 지표 충돌, 단일 tuple policy-fit이라는 열린 문제에 조언을 연결하는 편이 이 차수의 목적에 맞다.
독립 조건·장비 평가, 다양한 policy-fit 조건, 실측 source 비용, native AFA, 실제 장비의 순차 요청 실험은 남아 있다. 현재 결과만으로 robust adaptation, native benchmark 우월성 또는 물리 비용 절감을 주장하지 않는다. 아래에는 v16까지의 원문 해설을 보존한다.

ProbeFlow v16 해설: 좋은 모델을 먼저 고르면 센서 선택도 좋아지는가

v15의 부정 결과를 바꾸지 않고 다음 질문으로 간다

v15는 k=1·3·5·11·31을 실행했지만, 여덟 source mask의 selection 오류를 모두 합쳐 k 하나를 고르는 규칙에서는 k=1이 남았다. 이 결과는 그대로다. 다만 센서 예산이 정해진 상황에서 '모든 mask에서 평균적으로 좋은 k'와 '그 예산에서 실제 쓸 mask에 좋은 k'는 같은 문제가 아니다. v16은 이 차이를 강한 고정정책 비교로 실행했다. 더 똑똑한 AFA 모델을 학습한 연구가 아니다.
새 계산은 v15의 25,000개 저장 예측만 읽는다. selection 204 cycle과 이미 노출된 evaluation 421 cycle, 같은 한 장비, 같은 학습 bank, 같은 아홉 특징과 세 source를 유지한다. 새 예측·거리 계산·모델 적합·데이터 수집·adaptive rollout·모델 API 호출은 모두 0이다. 대신 새 고정정책 선택 2,160개,

공동 선택과 단계 선택의 대비 360개, evaluation 블록 진단 1,080개를 계산했다. 이 숫자는 새 독립 표본 수가 아니다.

두 최적화 순서는 왜 다를 수 있는가

단계 선택은 먼저 모든 mask에 걸친 component 오류 합으로 k를 고른다. 이후 그 k를 고정하고 주어진 예산에서 mask를 고른다. 공동 선택은 처음부터 가능한 k와 mask의 모든 쌍을 비교한다. 두 규칙은 같은 캐시와 selection 정보를 받을 수 있지만 서로 다른 목적을 최적화한다. 특히 단계 선택의 첫 평균에는 현재 예산으로 살 수 없는 mask도 들어간다.

공통 목적은 component 오류에 λ 곱하기 정규화 source quote를 더한 값이다. raw 가격표는 PS1·FS1·TS1에 6000·600·60, equal 가격표는 1·1·1을 부여한다. mask의 비용은 source별 quote 합이며 매 cycle의 hard budget을 넘을 수 없다. λ는 0·0.01·0.05·0.2·1을 미리 고정했다. 두 가격표의 단위가 다르므로 raw 차이와 equal 차이를 평균내 실제 비용 절감률로 해석하지 않는다.

공동 선택의 후보 집합에는 단계 선택이 고른 쌍도 들어 있다. 따라서 같은 selection 자료에서 공동 선택의 목적값이 더 나쁘지 않다는 것은 후보 집합 포함 관계에서 따라온다. 그 자체는 일반화 성능의 발견이 아니다. 결과를 모르는 새 자료로 넘어가서도 좋아지는지가 별도 질문이다. 이번 evaluation은 이미 과거에 노출되었으므로 그 질문의 독립 final이 될 수 없다.

기본 λ=0.05의 결과를 정확히 읽기

raw 가격표의 다섯 예산 조건은 전부 기존 선택을 유지했다. equal 가격표의 예산 1·2·3에서는 모두 같은 PS1을 선택하면서 k만 1에서 3으로 바뀌었다. 즉 아홉 조건 중 세 configuration이 달라졌지만 source mask 변화는 0개다. 이는 acquisition planner의 개선 증거가 아니라 예산별 predictor 선택 순서의 비교다. 세 예산은 동일 정책·동일 421 cycle을 반복하므로 독립적인 성공 세 번도 아니다.

PS1에서 selection 오류는 249/816에서 248/816으로 한 component만 줄었다. 이미 노출된 evaluation에서는 876/1684에서 799/1684로 77개 줄어 4.5724 percentage point가 낮아졌다. 네 head의 오류는 [65,273,246,292]에서 [0,273,246,280]으로 바뀌었다. 그러나 vector 오류는 둘 다 391/421이다. 일부 component가 더 맞아도 완전한 네 조건 진단은 개선되지 않았다.

더 강한 반대 비교도 남겨야 한다. 기존 TS1/k1 고정 control은 같은 equal quote 1에서 오류가 798/1684였다. 새 joint 선택의 799보다 한 component 적다. TS1의 selection 오류는 249/816이므로 joint가 그 결과를 보고 고르도록 바꾸지는 않는다. 이 비교는 새로운 최종 승자를 고르는 oracle이 아니라, 한 baseline을 이겼다고 전체 benchmark를 이겼다고 할 수 없음을 보여준다. 그 TS1 control의 수치는 기존 v15 실행에서 이미 얻은 값이며 새 예측으로 세지 않는다.

비용 가중치를 없애면 생기는 반례

λ=0에서는 공동 선택이 아홉 조건 중 다섯 configuration을 바꾸고, 그중 두 raw 조건에서 evaluation 오류가 오히려 늘었다. raw budget 6000에서는 TS1/k1 대신 PS1/k3를 택해 selection 오류를 1/816 줄였지만 실제 source quote가 60에서 6000으로 늘고 evaluation 오류는 1/1684 커졌다. raw budget 6660에서는 PS1+FS1/k5를 골라 quote가 60에서 6600으로 늘며 evaluation 오류도 75/1684 커졌다. 비용 항을 0으로 정한 실험에서는 hard cap을 지켜도 더 비싸고 나쁜 선택이 가능했다.

λ=0.01·0.05·0.2에서는 동일한 세 equal 조건만 바뀌고, λ=1에서는 어느 configuration도 바뀌지 않았다. 이 grid는 반복 개발 민감도 분석이다. 노출 evaluation에서 좋아 보이는 λ를 사후로 최종 채택했다고 쓰지 않는다. raw quote와 equal quote는 실제 측정 지연·현금·에너지 비용이 아니므로 물리적인 효율 주장으로 옮길 수도 없다.

시간 블록 검사는 무엇을 보여주고 무엇을 보장하지 않는가

selection 204개를 시간순 51개씩 네 블록으로 나눴다. leave-one-block-out은 153개로 선택하고 남은 51개를 본다. 이 방식은 남긴 과거 블록보다 나중 자료를 선택에 쓸 수도 있으므로 실시간 운용 재현이 아니라 블록 민감도 진단이다. 별도로 앞 51·102·153개만 보고 다음 51개를 보는 forward-prefix 진단 세 개도 실행했다. 여기에도 새 장비나 독립 이력이 생기지는 않는다.

equal budget1의 joint configuration은 leave-block 네 번 중 한 번, forward-prefix 세 번 중 두 번에서 전체 selection 선택과 달랐다. 한 leave-block에서는 PS1/k3 대신 FS1/k31이 선택됐다. 첫 51개와 첫 102개만 사용한 forward 선택은 PS1/k1을 남겼다. 1/816짜리 전체 selection 우위를 안정된 최적 모델의 인증으로 읽을 수 없는 이유다.
노출 evaluation의 네 블록 크기는 106·105·105·105다. 각 블록에서 sequential→joint component 오류는 282→251, 244→233, 205→195, 145→120이었지만 vector 오류는 각각 106·105·95·85로 변하지 않았다. 블록끼리 sampling independence가 있다는 가정이나 신뢰구간을 붙이지 않는다. 학습에 없던 정답 joint tuple의 비중도 selection100/204에서 evaluation391/421로 커졌다. 이는 알려진 label shift 진단이며 policy가 그 정답 stratum을 미리 보게 한 것은 아니다.

중요한 문헌을 어떻게 비교 계약에 반영했는가

BRiG-AFA v1 공식 HTML의 문제·방법·실험·한계 부분을 다시 확인했다. 그 논문의 동일 predictor 아래 정책 효과를 분리하는 관점을 따라, 여기서 k가 바뀐 차이를 acquisition planning 효과라고 부르지 않는다. unit-cost, fixed-horizon 설정과 우리의 source quote·hard cap도 다르므로 native BRiG-AFA를 구현하거나 성능 대결했다고 쓰지 않는다.
Janisch 등의 2024년 공식 publisher HTML에서는 비용 가중 목적과 hard budget의 구분, validation을 쓰는 선택 범위를 확인했다. 이를 따라 λ 민감도와 budget 위반을 따로 검사했다. 계층형 A2C 시스템을 재현한 것은 아니다. 논문들의 외부 성능 수치를 우리 표에 합치지 않았고, 전문 PDF를 다운로드하지 않았다. 새 문헌 확인 범위와 URL은 SOURCES.json 및 BENCHMARK_COMPARISON.json에 남겼다.

선택적으로 필요한 두 관점과 다음 중단 기준

baseline 비교 관점은 같은 training bank·입력 권한·예산·채점 단위를 지키면서 v15 단계 선택과 더 강한 공동 선택을 구별한다. 분포 관점은 노출 evaluation과 시간 블록 의존성, label support 변화, 다른 장비 데이터의 부재를 점검한다. 공개 방법론을 적용한 것이며 연구자 개인이 직접 자문한 것은 아니다. 새 표현학습을 하지 않았으므로 그 관점을 의례적으로 추가하지 않는다.
다음 확증은 새로운 조건 또는 독립 장비를 확보하고, source별 실제 quote와 predictor 선택 순서·λ를 그 final을 보기 전에 고정하는 작업이다. 그 전에는 '공동 선택이 현장에서 우월하다', '진단 coverage가 좋아졌다', 'adaptive acquisition을 입증했다'는 주장을 멈춘다. 현재의 진전은 어떤 baseline과 어떤 선택 계약을 비교해야 하는지 더 정확하게 만든 데 있다.

새 그림과 실행 경계

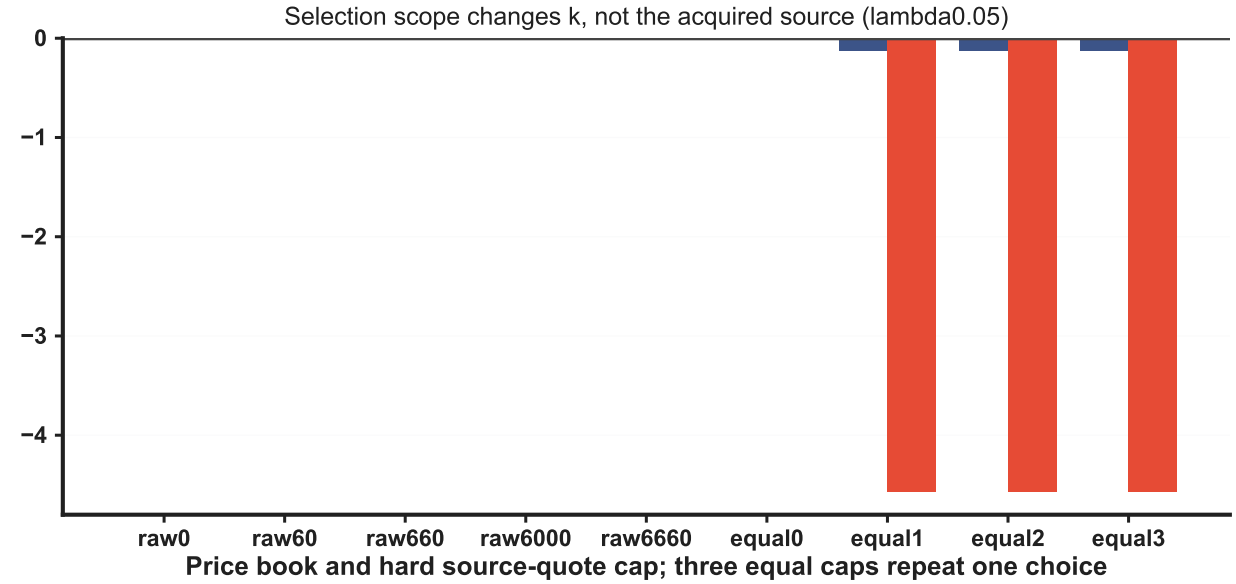


그림 3: 예산별 공동 선택과 단계 선택의 component 오류 차이, 기본 λ0.05

새 그림은 논문 그림 7이다. selection과 노출 evaluation의 각 분모를 보존하며, 같은 세 equal 예산이 동일 configuration을 반복한다는 점을 함께 읽어야 한다. analyze.py는 입력·계획·코드를 새 디렉터리에 먼저 동결했다. verify.py는 작성자의 분석 함수를 import하지 않고 25,000개 캐시, 2,160개 선택과 1,080개 블록 metric을 다시 계산했다. 같은 자료에 대한 대안 구현 검산이며 외부 과학적 독립 재현은 아니다.

아래는 v15까지의 원리·문헌·이전 결과를 그대로 보존한 배경이다. 그 안의 k1 유지와 개선0은 당시의 mask-평균 선택 규칙에 대한 결과다.

ProbeFlow v15 상세 해설: 새로운 조합을 만들 수 있어도 선택에서 이기는 것은 아니다

이번에는 실제 새 예측을 실행했다

v14까지는 동결된 1NN 예측을 다시 분석했다. v15에서는 head별 class-balanced kNN을 구현하고 새로컬 예측25,000개를 만들었다. 외부 모델 API는0회다. 새표준화나 optimizer 학습은 하지 않았고 정확한 v12 train-only bank·class count·means/scales를 재사용했다. 따라서 'API0이니 새예측0'이라고 쓰면 틀리다. 새 voting baseline1개 초기화, k설정5개, nonempty mask 거리query4,375개, empty fallback625개, 출력기록25,000개다.

train1..1540, selection1561..1764의204cycle, 이미 노출된 evaluation1785..2205의421cycle을 그대로 썼다. 사용 특징도 PS1/FS1/TS1 각각 평균·표준편차·기울기9개로 유지했다. mask가 허용한 채널의3 특징만 거리 계산에 공개한다. profile 정답, stable flag, cycle 번호와 평가용 지지집합 표시는 모델에 주지 않는다.

기존 1NN 상한을 넘을 수 있는 모델인가

1NN은 가장 가까운 한 train sample의4label tuple을 통째로 복사한다. 따라서 train에 없는4label 조합은 절대 정답으로 낼 수 없었다. 이번 모델은 k개 이웃을 고른 후 head마다 label표수를 해당 label

의 train빈도로 나눈 점수를 쓴다. 각 head에서 이 점수가 가장 큰 label을 고르므로 서로 다른 이웃의 label을 조합할 수 있다. 점수는 보정된 확률이 아니다.
예를 들어 이웃 joint label이(0,0),(0,1),(1,0)이고 두 head에서0이 아주 흔하고1이 드물면, 빈도를 보정한 투표는(1,1)을 고를 수 있다. train이웃에 없던 joint tuple이다. 이 최소 반례를 코드로 검사했다. 다만 주변 데이터가 지지하지 않는 정답을 잘 맞춘다는 뜻은 아니고, 출력 가능 집합이 넓어졌다는 뜻이다.
k=1에서는 이웃이 하나뿐이므로 빈도 보정과 무관하게 원래 tuple을 그대로 복사해야 한다. 실제로 기존1NN의5,000개 기록과 byte 수준의 tuple값이 모두 일치했다. aggregate가 우연히 같다는 약한 확인이 아니다. 이 검사가 실패했으면 동일 baseline 비교를 중단하도록 했다.

선택 결과는 개선 없음이다

k는1,3,5,11,31로 실행 전에 고정했다. selection의 모든8mask를 동일 가중한4head 오류합을 최소화하고 동률은 작은k로 정했다. selection오류합은 순서대로2215,2218,2225,2223,2216이었다. 분모는 204cycle×8mask×4head=6528비교다. 같은cycle을 반복 사용하므로6528개의 독립 표본이 아니다. 따라서 k=1이 선택된다. k31은1개 비교만 나쁘지만, 이것이 선택 안정성이나 유의한 차이를 보장하지 않는다. 선택된 모델은 기존과 같으므로421cycle×8mask의 모든 비교가 동률이고, 같은 비용·0.05 정규화 가격 계수의9가지 예산정책도 모두 같다. 선택으로 인정된 모델과 정책의 개선은0이다.
이미 노출된 평가 표를 보면 오류합은7058,6955,6871,6840,6817로 k가 큰 모델이 더 좋아 보인다. 이 표를 보고 k31로 바꾸면 evaluation정답으로 모델을 고르는 것이다. 이번 원고는 바꾸지 않는다. selection과노출evaluation의 괴리 자체가 개발 진단 결과다. final평가나 독립성능으로 표시하지 않는다.

그래도 상한의 전제가 중요한 이유

전체 원래5설정의 캐시25,000개를 읽는 보조 지지집합 감사를 별도계획으로 남겼다. 이 보조 감사는 첫 결과를 본 이후에 설계한 post-hoc 진단이고, 선택된k를 바꾸지 않으며 새예측도 만들지 않는다. train-seen/unseen 정답 strata는 모델에 주지 않는다.
선택되지 않은 k3, PS1+FS1(mask3)는 train에없는조합391cycle 중32cycle에서 train에없는tuple을 출력했고 그중10개를 정확히 맞혔다. 전체vector오류는391개에서381개로 줄어40/421정답이 된다. 기존 label-copy모델의30/421상한보다 크다. 이것은 이전 수학이 틀렸다는 뜻이 아니라, 새모델이 'train tuple만 복사한다'는 전제를 만족하지 않는다는 뜻이다.
다른진단으로 FS1만 쓴 k11은 train에없던정답14개를 만들고 전체vector오류392개를 보였다. 원래 FS1 k1은408개오류다. 하지만 이들설정은 selection에서 선택되지 않았고 그결과를 정책성과로 올리지 않는다. 특히 PS1+FS1 quote는raw6600/equal2이므로 raw6000이하예산에 들어가지 못한다. 비용조건을 빼고 성능만 더좋은정책이라고 해서는 안 된다.

왜 full source에서는 여전히 막히는가

모든k에서 full-source vector오류는391/421, head오류는[0,273,246,279]로 같다. train의cooler100 sample76개는 모두jointtuple(100,100,0,90)뿐이다. 개별head의 class가 있더라도 조건부조합과 거리구조가 제한되어 이웃이 유용한 다른head조합을 주지 못할 수 있다. 이것은 이번은행에서의 관찰이며 모든학습기의 불가능성 증명은 아니다.
출력공간 확장, 유용한 거리표현, 허용예산 내입력, selection이 이를 골라내는 능력, 독립final전이가 각각 다른관문이다. 어느하나를 만족했다고 다른관문을 자동통과시키지 않는다. 이번결과는 표현자유도가 일부정답으로실현될수있다는반례와, 정해진선택절차의개선0이라는부정결과를 동시에 남긴다.

필요한 다음 연구와 관점

다음에는 더많은 k를 평가표에맞춰 탐색하기보다 독립 조건·장비 또는 별도미노출final을 먼저확보해야 한다. 그뒤 head별거리/특징표현과강한compositional모델을 같은정보권한·가격·비용으로 비교한다. 적응적 acquisition을 복잡하게 만드는 것은 그 이후다. 문헌의native방법비교·물리비용·장기운영은 아직실행하지 않았다.

현재 검토에는 공정한 baseline 비교 관점과 분포 관점 두 개만 필요하다. baseline 관점은 같은 특징·분할·정규화·정보권한에서 k1 재현과 선택 고정을 검사한다. 이번에는 새 표현학습을 하지 않았으므로 그 기여를 전제로 한 관점을 억지로 할당하지 않는다. 분포 관점은 조건부 지지집합과 이미 노출된 자료의 한계를 검토한다. 공개 철학을 연결하는 것은 연구자 개인의 실제 자문이 아니다.

검증 범위

새 정량 그림은 논문 그림 6이다. 각 k의 component 오류 변화를 같은 partition의 k1에 대해서만 비교한다. selection에서 선택된 것은 k1이며, 노출 evaluation에서 더 좋은 막대를 보고 선택을 바꾸지 않는다.

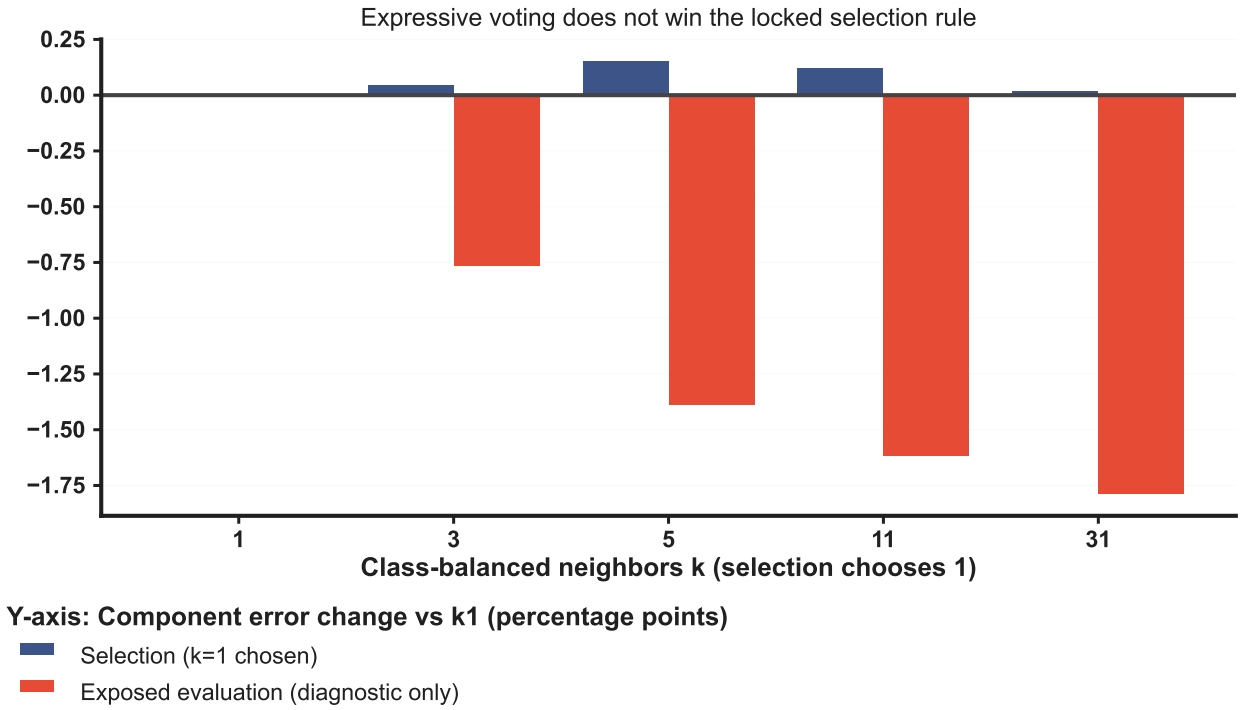


그림 4: 선택된 k1과 각 k의 selection 및 노출 evaluation 오류 변화

별도verify.py는 전체25,000기록의분모·고유cycle·head/vector요약을 재계산하고 k1의5,000기록재현을 다시 확인했다.6개결정적samplecycle의7mask×5k=210예측을 NumPy없이 원시특징에서 재계산했다. 전25,000예측의 이웃검색을 독립구현으로 다시 돌린것은 아니므로 그검산범위를 과장하지 않는다. 동일한 노출자료에대한 계산검산이며 새과학적 독립검증도 아니다.

아래는 v14까지의 상세 원리·문헌·이전 결정을 보존한 배경이다. v14에서 새fit/예측0이라고 적힌 문장은 이전 캐시분석의 범위다. v15는 위와 같이 새예측25,000개를 실행했다.

ProbeFlow v14 해설서

v14는 보존된 예측에서 무슨 오류를 줄일지 바꾸면 선택도 달라지는가, 그리고 예측기가 내놓을 수 없는 완전 상태는 무엇인가를 실제로 계산한 후속 연구다. 기존 1–15절의 v11–v13 설명과 부정 결과를 보존하고, 새 16–24절에 목적별 선택·후회·Pareto·출력 지원 상한·재현 절차를 추가했다. 새 분석을 실행했지만 새 예측·학습·적응형 rollout·물리 실험·독립 최종 평가를 한 것은 아니다. 기존 평가 구간의 노출도 그대로다.

먼저 알아야 할 결론과 증거의 범위

ProbeFlow의 현재 핵심은 "더 멀리 계획하는 정책이 더 좋다"가 아니다. 정보를 획득한 뒤 판단하는 예측기가 그 정보를 쓸 수 있는지부터 확인해야 정책 비교를 해석할 수 있다는 문제 제기다. 실제 공공 센서 자료에서 이 순서를 따라가자, 잘못된 학습 분할을 고쳐도 전체 진단이 회복되지 않았고, 예측기 점수식의 작은 수정으로 좋아진 성능도 네 개 목표 중 하나에만 집중되어 있었다. 간단한 센서

부분집합은 비싼 전체 입력과 동률이었지만 다른 가격 체계에서는 선택 절차 자체가 뒤의 구간에 잘 옮겨가지 않았다.
이 문서는 영어 원고의 논리, 수식, 비교 대상, 그림을 깊게 읽기 위한 해설이다. v13에서는 새 센서 실험을 추가하지 않았다. 그 수치는 보존된 v11·v12 실행에서 왔으며, v14는 같은 예측에서 목적별 선택·오류 교환·출력 상한을 새로 계산했다. 문헌은 명시된 범위의 공식 1차 HTML 페이지를 직접 확인했다. 논문 전체의 모든 증명, 저자 코드, 외부 성능표를 검증한 것은 아니다. 참고문헌의 번호와 URL, 확인한 절, 확인하지 않은 범위는 paper.json과 SOURCES.json에 있다. 저자들이 보고한 외부 성능과 로컬 실측은 합치지 않는다.
데이터를 처음 보는 최종 시험이라는 해석도 허용하지 않는다. 평가 구간 421개 사이클은 v9와 v11에서 이미 관찰되었다. v12에서 계획과 선택 결과를 먼저 고정한 것은 현재 실행의 절차를 재현할 수 있다는 뜻이다. 과거 관찰을 지워주지는 않는다. 물리적으로 독립인 설비·실험 세션의 수는 알려져 있지 않다.

1. Introduction은 왜 정책보다 예측기를 먼저 묻는가

능동적 특징 획득(active feature acquisition, AFA)은 처음부터 모든 입력을 무료로 보는 대신, 필요한 정보를 순서대로 요청하는 문제다. 예를 들어 압력·유량·온도 중 무엇을 읽을지, 이미 읽은 정보로 판단을 끝낼지, 남은 예산을 써서 하나 더 읽을지를 결정한다. 여기에는 두 개의 서로 다른 기계가 있다. 예측기는 현재 보이는 정보에서 상태를 추정한다. 획득 정책은 다음에 어떤 정보를 보여줄지 정한다.
정책이 좋은 정보를 골라도 예측기가 그것을 활용하지 못하면 최종 오차는 줄지 않는다. 예측기가 어느 센서 조합에서도 같은 답을 낸다면, 정책의 복잡도를 올려도 그 답을 바꾸는 경로가 없을 수 있다. 판단을 보류하는 문턱 때문에 모든 답이 거절된다면, 획득 순서의 변화는 실제로 채택된 진단을 만들지 못할 수 있다. 따라서 정보 선택의 가치를 말하기 전에 "무엇을 선택해도 예측 결과가 같은가", "전체 정보를 줘도 유용한가", "실제 채택된 판단은 몇 개인가"를 구분해야 한다.
그때 줄일 수 있는 낭비는 구체적이다. 탐색 깊이를 늘리면 미래 관측 분포의 근사, 후보별 예측 호출, 탐색 트리, 하이퍼파라미터, 비용 기록, 누수 검사가 추가된다. 예측기가 작동하지 않는 단계에서는 이 작업이 유용한 진단을 입증하지 못할 가능성이 크다. 본 연구에서는 먼저 작고 완전히 열거 가능한 예측기·센서 마스크 비교를 수행해 그 위험을 조사했다. 다만 이런 연구 순서를 따르는 팀이 실제로 더 빨리 성공한다는 인과 실험은 하지 않았다. "연구 생산성 개선이 입증되었다"라고 쓰면 현재 근거를 넘는다.

그림 1 읽기: 실패가 다음 실험을 어떻게 바꾸었는가

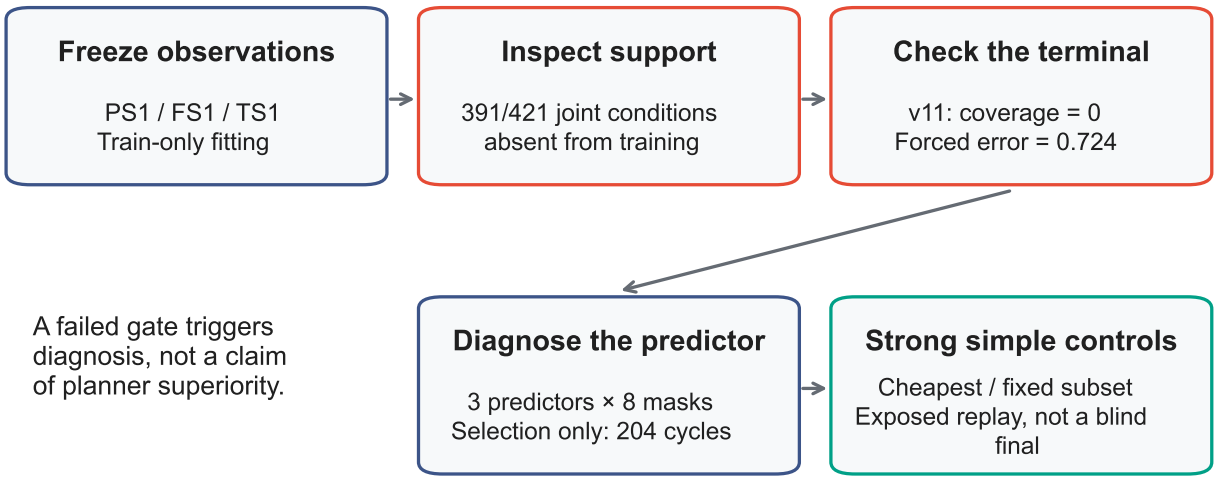


그림 5: 그림1 설명

그림의 첫 단계는 자료와 정보 권한을 고정하는 것이다. 그다음은 라벨 지원을 확인한다. v9에서는 평가 cooler 클래스가 학습에 없었다. 학습 접두 구간을 늘리자 각 목표의 클래스는 학습에 나타났지만, 네 목표를 함께 묶은 조건은 여전히 대부분 지원되지 않았다. 그 상태에서 v11의 선택적 진단은 수용률 0이었다. 따라서 다음 행동을 더 깊은 계획기로 정하지 않고, 예측기 세 개와 센서 부분집합 여덟 개의 고정 비교로 바꾸었다.
화살표를 "모든 조건을 통과했다"로 읽으면 안 된다. 지원 문제는 남아 있고, 그림은 현재 연구가 어떤 질문을 먼저 해결하려 했는지를 보여준다. 마지막의 획득 비교도 작은 고정 부분집합 비교까지다. 겹치는 데이터 원천, 부분 통계 공개, 실패한 요청의 캐시 등 더 일반적인 실행기 상태의 효과는 아직 검증되지 않았다.

2. Related Work를 읽는 세 가지 층위

문헌 비교에서는 이름이 비슷한 알고리즘을 가져와 같은 것으로 부르는 실수를 피해야 한다. 현재 원고는 세 층위를 구분한다.

층위	무엇을 보존하는가	현재 ProbeFlow의 위치
원형 재현(native reproduction)	저자 알고리즘, 학습 절차, 예측기, 목표, 데이터·평가 계약	수행 0건
공통 계약 어댑터	센서·가격·예산 등을 공통으로 바꾸되 변경점을 명시	향후 수행할 비교
직접 만든 참조 정책	단순한 고정 순서, 부분집합, 경험적 탐색	v11·v12 실측의 실제 위치

원형 재현은 원 논문의 수치와 비교하기 쉽지만, 데이터와 비용 구조가 달라 공정한 공통 비교가 어려울 수 있다. 어댑터는 공통 문제에서 비교하기 쉽지만, 원 논문의 핵심을 바꿀 수 있다. 두 결과는 별도로 필요하다. v11의 경험적 두 단계 탐색에 L2M이라는 이름을 붙이면 원 논문의 예측기·학습·목표 차이를 숨기게 된다.

기존 AFA가 이미 다루는 것

Shim 등의 연구 [8]는 획득과 분류를 함께 학습하며, 관측 특징의 수가 달라지는 표현과 비용을 고려한 행동을 다룬다. EDDI [12]는 부분 관측을 다루는 생성 모형과 정보 이득을 사용한다. 생성 대리모형을 쓰는 AFA [9] 역시 아직 읽지 않은 특징의 가능한 값을 모델링하는 길을 제시한다. 이들의 고수준 목적을 확인한 범위는 공식 초록이다. 세부 구조나 실제 성능을 이번 실험에서 재현하지 않았다.
Covert 등의 연구 [10]는 조건부 상호정보량을 활용하는 학습된 탐욕적 선택을 비교 대상으로 생각하게 한다. 여기서 가져오는 교훈은 "탐욕적 방법이면 약하다"는 전제를 두지 말자는 것이다. 현재 세 센서는 여덟 부분집합을 모두 조사할 수 있으므로, 간단한 부분집합 선택도 강한 비교 대상이다. 학습된 탐욕적 정책과 현재의 고정 부분집합은 다른 방법이므로 같은 구현으로 취급하지 않는다.

최근 비근시안적 방법 네 편의 정확한 역할

참고문헌	확인한 핵심 설계	우리가 수행하지 않은 것
Learning-To-Measure [2]	인컨텍스트 예측기와 획득, 탐욕적·미래 탐색 변형	원형 메타 예측기, 학습, 원래 평가
BRiG-AFA [3]	고정 예측기에서 남은 예산별 미래 위험 학습	Bellman 목표의 신경망 적합과 원형 재현
O-MPAC [4]	묶음 원천 획득과 오프라인 계획에서 학습한 정책	원형 교사·점수기 학습과 실험
NM-PPG [5]	경로 미분에 기반한 비근시안적 궤적 최적화	경사 추정, 원형 학습·벤치마크

네 문헌의 해당 arXiv 버전과 1차 HTML 내용은 이번 재편에서 확인했다. 현재 날짜에서 미래에 있는 것으로 간주해 임의로 삭제하지 않았다. 그러나 확인된 서지가 알고리즘의 우수성을 인증하지는 않는다. 이 문헌들 때문에 "여러 단계 앞을 본다", "예측기는 고정한다", "센서 묶음을 요청한다" 자체는 새 기여가 될 수 없다. ProbeFlow의 현재 기여는 이 방법들보다 나은 최적화기가 아니라, 이들을 비교하기 전에 무엇이 이미 실패했는지 드러내는 실행 가능한 진단이다.

선택적 분류와 오프라인 평가가 별도의 문헌인 이유

선택적 분류 [18]는 틀릴 위험과 답하는 비율을 함께 본다. SelectiveNet [17]는 예측과 거절을 함께 학습하는 비교 방향을 보여준다. 현재 v11은 그런 모델이 아니라 고정 예측기에 개발 구간 문턱을 붙인 시스템이다. SGR [6]는 명시된 표본·탐색 가정 아래 위험을 제어하는 구성이며, 우리 문턱은 그 인증을 갖지 않는다.

획득 정책이 바뀌면 관측되는 자료의 분포도 달라질 수 있다. 시간 변화 특징의 AFA 평가를 다룬 연구 [11]는 이 연결을 따져야 한다는 근거다. 현재 완전 기록에서 마스크를 씌우는 재생은 구현된 고정 부분집합에 대해서만 좁게 해석한다. 관찰되지 않은 적응적 경로 전체에 대한 일반적인 오프라인 평가 보장으로 확대하지 않는다.

3. Problem and Diagnostic Method: 상태·행동·정보 권한

한 사이클에는 전체 센서 기록과 실제 상태 라벨이 있다. 정책은 라벨을 보지 않는다. 시작할 때 받는 것은 센서 목록, 가격, 남은 예산, 허용 행동, 공개된 이력이다. 센서 요청이 성공하면 그 요청으로 허용된 특징을 받는다. 아직 요청하지 않은 평가 센서값이나 미래 센서의 실제 결과를 정책에 미리 주면 정보 누수다.

일반형에서는 실행기가 내부적으로 해 둔 작업과 정책에 공개된 정보를 구분해야 한다. 예를 들어 원시 신호를 캐시에 읽어 놓았어도 평균만 공개했다면 표준편차까지 정책이 아는 것은 아니다. 원천을 다시 읽을 필요가 없어 한계 비용이 낮아질 수는 있어도, 공개하지 않은 수치를 임의로 사용할 권한은 생기지 않는다. 이는 연구 계약의 요구사항이고, 현재 실측에서 입증한 새로운 상태 표현의 우수성은 아니다.

이번 세 센서 실험은 더 단순하다. 센서 하나를 요청하면 미리 정한 평균·모표준편차·기울기가 한꺼번에 공개된다. 요청은 결정론적으로 성공하고 센서 가격도 고정이다. 따라서 획득한 센서 마스크와 공개 값, 남은 예산, 센서 목록으로 구현된 정책을 설명할 수 있다. 별도의 숨은 캐시 변수를 추가해도 효과가 있다는 주장은 하지 않는다.

POMDP라는 관점은 어떤 정보가 보이지 않는지, 행동이 무엇을 공개하는지, 보상과 종료가 무엇인지를 명시하는 데 도움이 된다 [13]. 이 명칭만 붙였다고 최적 정책을 풀었거나 상태 표현이 충분함을 증명한 것은 아니다. 현재는 그 일반 문제의 매우 작은 결정론적 부분집합을 실행했다.

현재 확신과 미래 관측 모형은 다르다

현재 예측기의 확신 점수가 낮다는 사실만으로 다음 센서를 읽었을 때 어떤 값이 나올지는 알 수 없다. 미래 탐색은 센서의 가능한 결과와 그 확률을 필요로 한다. v11에서는 학습 자료의 완전 기록을 이용한 경험적 대리 분포로 이를 근사했다. 요청하지 않은 평가 센서의 실제 값은 미래 예측으로 사용하지 않았다. 다만 학습과 평가의 공동 조건이 크게 다르면 이 근사 역시 부정확할 수 있다.

v12가 마스크 비교로 돌아온 이유는 이 두 실패를 분리하기 위해서다. 좋은 미래 분포가 없어서 정책이 실패한 것인지, 주어진 관측을 예측기가 활용하지 못해 실패한 것인지를 처음부터 복잡한 계획기 하나로 동시에 확인하기 어렵다. 모든 마스크를 열거하면 적어도 고정 예측기가 부분 입력을 받았을 때 무엇을 하는지 직접 볼 수 있다.

4. 비용 수식: 공유 원천·경성 예산·연성 벌점

획득 행동 집합을 A, 실제로 요청한 원천들의 집합을 G(A), 원천별 가격을 c_g라 하면, 현재 논리적 획득 견적은 다음과 같다.

C(A) = sum_{g in G(A)} c_g

같은 센서에서 평균·표준편차·기울기 세 개를 얻어도 원시 신호 읽기는 한 번으로 센다. 통계마다 원천 가격을 세 번 청구하면 공유 구조를 잃는다. 반대로 내부 캐시가 있다는 이유로 읽지 않은 다른 센서 특징까지 무료로 공개하면 권한을 어긴다.

현재 가격 체계는 두 개다.

체계	PS1	FS1	TS1	전체 원천 견적	뜻
원시 표본 대리 가격	6000	600	60	6660	사이클당 원시 표본 수를 이용한 논리적 대리값
동일 원천 가격	1	1	1	3	센서 하나를 같은 비용으로 가정

서로 다른 단위를 더하지 않는다. 또한 원시 표본 수는 실제 전력·통신량·파일 읽기 지연·돈을 측정한 값이 아니다. 평가기는 모든 마스크의 결과를 계산하려고 전체 특징 은행을 내부에서 읽어 둔다. 견적은 각 정책에 허용한 공개 정보의 장부이며, 실제 컴퓨터가 어떤 파일을 전혀 읽지 않았다는 증거가 아니다.

경성 예산 B는 C(A) <= B라는 실행 허용 조건이다. 연성 목적은 component_error + 0.05 × C(A) / C(all_sources)다. 후자는 오차와 비용의 절충 순위를 만들지만 경성 예산을 넘을 권한을 주지 않는다.

계수 0.05와 정규화는 현재 결과를 본 뒤 바꾸지 않았다. 두 가격 체계에서 같은 숫자의 예산을 직접 비교하는 것도 허용하지 않는다.
v11의 일부 전체 집계 필드는 서로 다른 가격 단위를 합친 오류가 있었다. 원본 결과는 보존했고 단위별 재집계를 별도 기록으로 남겼다. 정책별 예측·견적은 바뀌지 않았다. 원고는 이 사건을 재현 부록에 두었다. 해시가 일치해도 물리 단위가 틀린 합은 옳아지지 않는다는 점이 핵심이다.

5. Experimental Setup: 시간 분할과 지원의 두 의미

자료는 UCI Hydraulic의 로컬 캐시 중 PS1·FS1·TS1 세 채널이다 [1]. 사이클마다 이미 종료된 60초 신호에서 평균, 모표준편차, 시간축을 정규화한 선형 기울기를 구한다. 총 아홉 특징이다. 라벨, 안정성 플래그, 사이클 번호를 예측 특징으로 넣지 않았다. 결과는 완료된 기록의 분류에 관한 것이고, 실시간 조기 탐지의 지연을 측정한 것이 아니다.

구간	사이클 번호	개수	허용 작업
학습	1–1540	1540	스케일러, 중심, 최근접 이웃 참조 은행
제외 간격	1541–1560	20	사용하지 않음
선택	1561–1764	204	사전 정의 모델 선택·부분집합 선택
제외 간격	1765–1784	20	사용하지 않음
노출된 평가	1785–2205	421	기술적 오차 비교

시간 순서를 유지한 것은 임의 분할로 어려운 구간을 섞지 않기 위해서다. 하지만 20개를 띄웠다고 물리적으로 독립이 되는 것은 아니다. 구조를 고려한 검증은 배포 목표와 맞아야 하고, 차단은 새 조건으로의 외삽 문제를 만들 수도 있다 [16]. 세션·설비 ID가 없으므로 현재는 독립 단위 수를 정하지 않았다.

주변 라벨 지원과 공동 조건 지원

v9 학습은 1323에서 끝나 cooler=100을 전혀 포함하지 않았다. 평가 421개는 모두 cooler=100이었다. 폐쇄 클래스 예측의 출발 조건부터 맞지 않은 셈이다. 접두 학습을 1540까지 늘리자 cooler=3,20,100이 각각 732,732,76개가 되었다. 각 목표별로는 평가 클래스가 모두 학습에 등장한다.
그러나 cooler=100인 학습 76개는 네 목표를 묶으면 모두 (100,100,0,90)이다. 평가에서 이 튜플은 30번만 나타난다. 나머지 391개, 즉 약 92.87%는 학습에 없던 공동 조건이다. 이는 "모든 클래스가 있었다"와 "진단 조합을 배웠다"가 다른 말임을 보여준다.
튜플이 없다는 것은 관찰된 라벨 조합이 학습 표에 없다는 정확한 기술이다. 어떤 물리 인과 메커니즘이 새롭다거나 그 조합의 예측이 원리적으로 불가능하다는 증명은 아니다. 전체 프로파일의 같은 조건 연속 구간 194개도 독립 물리 실험이라고 확인되지 않았다. 이들을 독립 표본으로 바꾸어 신뢰 구간을 만들지 않는다.

6. 예측기 세 개를 왜 이렇게 골랐는가

세 후보는 새 모델을 끝없이 찾기 위한 목록이 아니다. 사전 고정한 작은 진단이다. 가장 단순한 점수식 변경으로 무엇이 달라지는지와 국소적인 참조 규칙으로 무엇이 달라지는지를 나누어 본다.

기존 사전확률 포함 중심 예측기

각 목표의 클래스 중심까지 공개된 특징 차원의 표준화 제곱 거리를 계산하고, 학습 클래스 비율의 로그 항을 포함한다. 스케일러·클래스 중심·비율은 모두 학습 1540개만 쓴다. 선택·평가 통계를 스케일러에 넣지 않는다.
현재 구현의 거리와 사전확률 항은 점수식의 정확한 스케일에 영향을 받는다. 다른 논문의 Bayes 분류기와 같다고 부르지 않는다. v12의 목적은 이 코드에서 사전확률 항을 제거했을 때 무엇이 바뀌는지 확인하는 것이다.

같은 중심에서 사전확률 항만 제거

비어 있지 않은 마스크에서 로그 사전확률 항만 없앤다. 중심·스케일·거리·자료는 같다. 이 비교는 코드 안에서 하나의 점수 항을 바꾼 통제 비교다. 산업 모집단에서 균등 사전확률이 맞다거나 실제 고장 원인을 식별했다는 뜻은 아니다.

빈 마스크는 세 후보 모두 목표별 학습 최빈 클래스를 사용한다. 아무 특징도 없는데 중심 거리의 차이로 답이 달라지는 것을 막고 공통 출발점을 유지한다. 동률 규칙도 고정되어 있다.

학습 전용 1-최근접 이웃

표준화된 공개 특징 차원만으로 학습 1540개 중 가장 가까운 행을 찾고 그 행의 네 목표를 함께 반환한다. 정확한 거리 동률이면 가장 이른 학습 행을 고른다. 행 번호는 동률 규칙이지 특징이 아니다. k, 거리, 스케일러를 탐색하지 않는다.

이 후보는 중심과 다른 국소 구조를 쓰지만 동시에 네 목표를 묶어 반환하는 방식과 귀납적 편향도 바꾼다. 따라서 성능 변화 하나로 "표현이 원인"이라고 분리할 수 없다. 사전확률 제거와 달리 한 가지 성분만 바꾼 실험이 아니다.

선택 권한과 유용성 관문

세 센서의 모든 부분집합은 빈 집합을 포함해 여덟 개다. 각 모델의 선택 구간 오차를 여덟 마스크에 같은 가중치로 평균하고 가장 낮은 모델 하나를 고른다. 전체 센서 하나만 보고 고르지 않는 이유는 획득 시스템이 실제로 부분 입력을 사용하기 때문이다. 마스크 가중치가 실제 현장 사용 비율을 추정한 값은 아니다.

선택된 모델은 전체 입력의 선택 오차가 빈 입력보다 최소 0.05 작아야 후속 정책 대조에 들어간다. 틀린 형식의 출력도 없어야 한다. 통과하지 못하면 추가 정책 탐색을 하지 않는다는 사전 중단 규칙이다. 이 0.05는 통계적 유의성 문턱이나 위험 보장이 아니다.

선택 평균은 기존 중심 0.555147, 사전확률 제거 중심 0.348499, 1NN 0.339308이다. 1NN이 선택되었다. 선택 전체 입력 오차 0.305147은 빈 입력 0.555147보다 0.25 낮아 관문을 통과했다. 모델과 부분집합이 같은 선택 라벨 204개를 재사용했으므로 개발 과적합의 여지가 있다 [14,15]. 이 사실을 숨기기 위해 이미 노출된 평가를 독립 검증으로 다시 이름 붙이지 않는다.

7. 성능 지표를 어떻게 계산하는가

각 사이클마다 cooler·valve·pump·accumulator 네 판단이 있다. component error는 네 목표의 틀린 횟수 합을 4 × 사이클 수로 나눈다. 평가에서는 분모가 1684다. vector error는 네 개 중 하나라도 틀린 사이클 수를 421로 나눈다. 동일한 분모가 아니며, 앞 지표가 좋아도 완전한 진단은 거의 틀릴 수 있다.

선택적 분류에서는 coverage를 수용한 사이클 수 / 전체 사이클 수로 계산한다. selective error는 수용한 판단만으로 오차를 계산한다. 수용한 판단이 0이면 이 비율은 0이 아니라 정의되지 않는다. 결과 JSON에서는 null이다. 보류 손실 0.3은 별도의 종단 손실이며, selective error로 바꾸어 읽으면 안 된다.

세 모델 × 여덟 마스크 × 421개는 평가 예측 기록 10,104행이다. 독립한 새 사이클 10,104개가 아니다. 선택은 3 × 8 × 204 = 4896행이다. 정책 36행도 같은 마스크 예측의 조회 결과다. 상관된 여러 표현을 독립 반복으로 세면 불확실성을 과도하게 줄일 수 있다. 현재는 독립 물리 단위가 없어 모집단 신뢰구간이나 유의확률을 보고하지 않는다.

8. Results: v11의 실패를 v12가 지우지 않는 이유

v11에서 선택 문턱은 1.01이었고 정책·예산·가격 조합 47행 모두 coverage=0이었다. selective error는 null, 보류 종단 손실은 0.3이다. 클래스 지원을 고쳤다는 것만으로 선택적 진단이 살아나지 않았다.

이후 별도의 사후 진단 한 번에서 문턱을 0으로 강제했다. 원래 선택적 결과를 갈아끼우지 않았다. 원시 견적 예산 6660에서 경험적 탐욕 정책의 평균 견적은 약 269.50, 두 단계 탐색은 약 2419.38이었다. 모두 component error 0.723872, vector error 1이었다. 더 오래 탐색한 정책이 더 많이 읽었지만 이 예측기의 진단은 나아지지 않았다.

계획과 재생의 한 번 측정 시간은 탐욕 약 2.48초, 두 단계 약 23.03초, 중단 약 0.015초였다. 순차 실행의 캐시 영향 등이 통제되지 않았으므로 정밀 CPU 벤치마크나 실제 센서 시간으로 해석하지 않는다. 이 비교가 말하는 것은 해당 실행에서 탐색 확장이 예측 개선을 주지 않았다는 것이다. 비근시안적 정책 일반의 무용성을 입증하지 않는다.

9. Results: 25%p 개선의 정확한 내용

전체 입력 평가에서 기존 중심 오차는 0.723872다. 사전확률 항을 제거하면 0.473872로 낮아진다. 선택된 1NN도 동일하다. 표면적으로 25%포인트는 커 보이지만 목표별 오류를 보면 의미가 달라진다.

전체 입력 예측기	cooler 오류	valve 오류	pump 오류	accumulator 오류	component error
기존 중심	421	273	246	279	0.723872
사전확률 제거 중심	0	273	246	279	0.473872
선택된 1NN	0	273	246	279	0.473872

421개의 cooler 판단만 전부 뒤집혀 맞았으므로 전체 네 판단의 평균에서 정확히 1/4=0.25가 줄었다. 다른 세 목표는 회복되지 않았다. 1NN은 전체 입력에서 모든 평가 사이클을 (100,100,0,90)으로 예측한다. 완전 일치는 30개뿐이라 vector error는 391/421=0.928741다. 평균 component error만 보여주면 이 단순한 상수 튜플의 한계가 보이지 않는다.

같은 중심을 유지하고 점수 항만 제거한 비교는 이 데이터·코드에서 그 항을 바꾸는 것으로 cooler 변화가 충분히 생겼음을 보여준다. 하지만 원래 자료의 물리적 고장 메커니즘을 식별한 것은 아니다. 1NN과 단순 중심의 전체 입력 동률은 그 개선을 특별한 최근접 이웃 표현의 독립 효과로 돌릴 수 없게 한다.

그림 2 읽기: 마스크 전체를 봐야 하는 이유

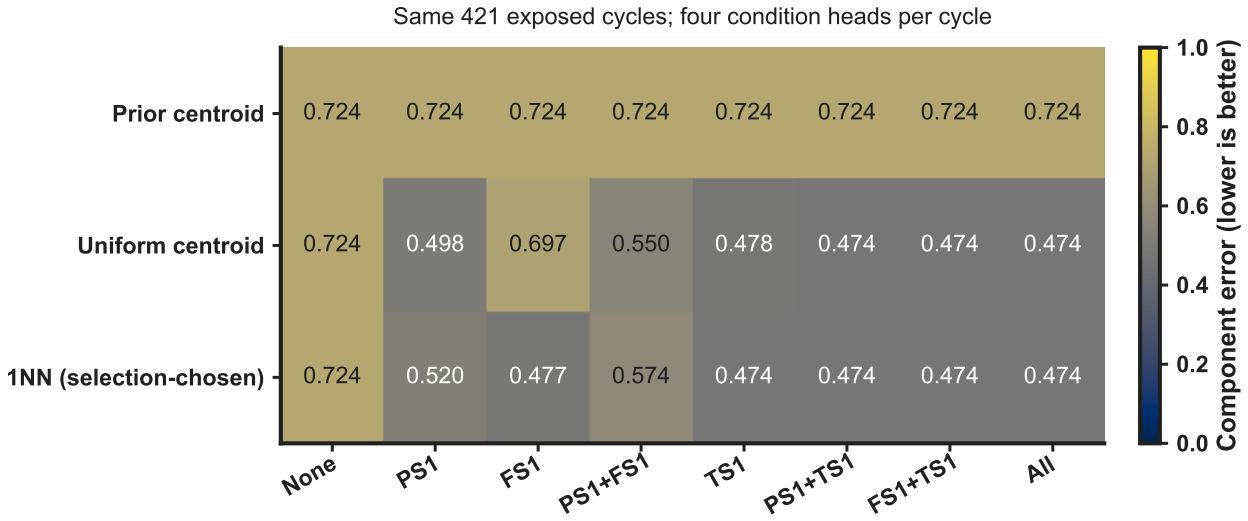


그림 6: 그림2 설명

행은 세 예측기이고 열은 같은 여덟 센서 마스크다. 색과 숫자는 같은 421개 노출 평가 사이클의 평균 네 목표 오차다. 세 행을 더 많은 독립 실험으로 읽지 않는다. 1NN은 그림의 평가 수치를 보고 뽑은 모델이 아니라 선택 204개의 사전 규칙으로 뽑혔다.

마스크	기존 중심	사전확률 제거	1NN
없음	0.723872	0.723872	0.723872
PS1	0.723872	0.498219	0.520190
FS1	0.723872	0.696556	0.477435
PS1+FS1	0.723872	0.550475	0.574228
TS1	0.723872	0.478029	0.473872
PS1+TS1	0.723872	0.473872	0.473872
FS1+TS1	0.723872	0.473872	0.473872
전체	0.723872	0.473872	0.473872

중요한 반례는 1NN의 FS1에서 PS1+FS1로 가는 변화다. 입력이 늘지만 오차는 약 0.477에서 0.574로 나빠진다. 이 규칙은 좌표가 늘면 가까운 학습 이웃이 바뀌고, 해로운 차원을 학습해서 무시하는 선택권이 없다. 따라서 전체 입력을 위험의 하한이나 정확도의 상한으로 가정하지 않는다. 최적화된

이상적 의사결정자의 정보 가치에 관한 일반 명제를 반박한 것이 아니라, 고정된 부정확한 예측기의 행동을 측정한 것이다.
비슷한 평균도 서로 다른 오류일 수 있다. FS1의 목표별 오류는 398,153,51,202이고 TS1은 0,273,246,279다. 펌프를 중요하게 보는 현장의 효용은 단순 4등분 평균과 다를 수 있다. 현재는 산업 손실을 조사하지 않았으므로 어떤 센서가 실제로 더 안전하다고 결론내리지 않는다.

10. Results: 간단한 정책은 왜 반드시 남겨야 하는가

정책 네 개는 같은 선택된 1NN을 쓴다. 중단은 아무것도 읽지 않는다. cheapest-first는 가격순, 동률이면 고정된 사전식 순서를 쓴다. pressure-first는 PS1→FS1→TS1 순서로 예산 안에서 읽는다. selection-locked subset은 여덟 부분집합 중 가능한 것만 비교해 선택 구간 오차와 고정 비용 벌점의 합이 가장 작은 것을 고른다. 동률이면 더 싼 것, 그다음 더 작은 마스크 번호를 고른다.
마지막 정책은 센서가 세 개라 가능한 강한 단순 기준이다. 평가 라벨을 보고 최고의 부분집합을 고르는 oracle이 아니다. 선택 라벨을 사용한 감독형 개발 기준이므로, 아무 개발 라벨도 못 쓰는 정책과 자원 차이를 감춘 채 비교해서도 안 된다.

그림 3 읽기: 지원 문제와 가격별 결과를 함께 보기

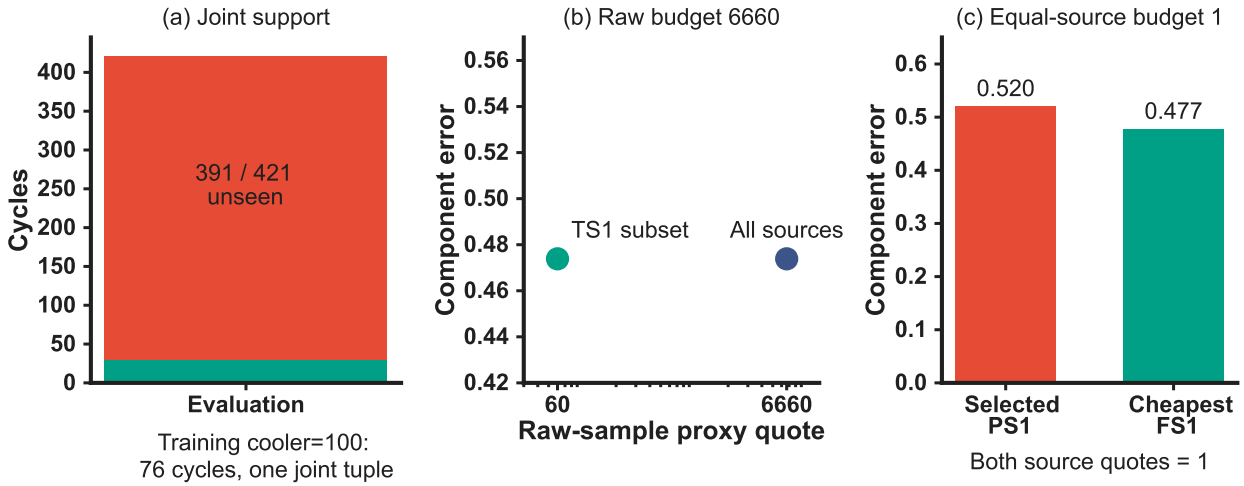


그림 7: 그림3 설명

첫 패널은 주변 클래스의 존재와 공동 조건의 부재를 구분한다. 나머지 두 패널은 서로 다른 가격 단위를 사용하므로 축을 하나로 합치지 않는다. 원시 가격 예산 6660에서 선택 부분집합 TS1은 견적 60으로 전체 센서 견적 6660과 같은 0.473872 오차를 낸다. 비용이 저렴한 단순 정책과 동률인 이상 더 복잡한 정책의 이득을 먼저 주장하기 어렵다.
동일 원천 가격 예산 1에서는 방향이 바뀐다. 선택 구간에서 여러 센서 마스크가 동률이었고 고정 마스크 번호 규칙으로 PS1이 선택되었다. 이 정책의 평가 오차 0.520190은 cheapest-first가 고른 FS1의 0.477435보다 나쁘다. 두 정책의 견적은 모두 1이다. 예산 3에서도 선택 정책은 PS1을 유지하지만 전체 센서는 0.473872다. 그 구간에서는 비용 벌점까지 포함한 목적도 전체 센서 쪽이 낫다.
이것은 결과를 본 다음 동률 규칙을 TS1 쪽으로 바꾸고 싶은 유혹을 만든다. 현재 연구는 그렇게 하지 않았다. 선택 단계의 동률이 다음 조건 구간에서 동등한 성능을 뜻하지 않는다는 실패를 보존한다. 가격 체계가 바뀌면 허용 부분집합과 동률 선택이 달라진다는 점도 별개로 남긴다. 어떤 새 정책을 붙이든 이 실패보다 더 잘 일반화하는지와 개발 권한이 같은지를 검토해야 한다.

11. Discussion: 가설과 반증을 짝짓는 법

현재 결과에서 가능한 설명을 다음과 같이 나누면 과잉 해석을 줄일 수 있다.

설명 후보	현재 자료가 보여주는 것	아직 필요한 반증 가능 비교
사전확률 점수 항이 cooler 예측을 억제	같은 중심에서 항만 제거하면 변화	새로 지원되는 조건·설비에서 동일 변경의 전이

표현 또는 국소 구조가 병목	1NN이 선택 평균에서 더 낮음	다른 가정을 고정한 표현 대조, 독립 구간
더 깊은 계획이 필요	v11에서는 깊은 계획도 개선 없음	유용한 예측기·미래 모형을 갖춘 공통 계약 비교
TS1이면 충분	현재 노출 구간의 component 오차가 전체와 동률	목표별 산업 손실·다른 조건·다른 설비에서 검증
캐시 상태 표현이 기여	현재는 권한 계약만 정의	이력으로 복원하는 기준과 부분 공개 실험

원고의 반증 절은 실패를 미래 논문의 장식으로만 쓰지 않기 위한 장치다. 강한 단순 예측기를 넣었더니 정책 이득이 사라지면 정책 특유의 기여가 약해진다. 원천 ID와 정보 공개를 바로잡았더니 비용 이득이 사라지면 그것은 장부 또는 누수의 효과일 수 있다. 이력이 캐시 상태를 복원하면 더 복잡한 숨은 상태가 불필요할 수 있다.

반대로 현재처럼 좁고 정확한 기술적 주장을 하는 보고서에 산업 일반화가 없다는 이유로 이미 측정한 수치까지 부정할 필요는 없다. 무엇이 측정되었고 무엇을 주장하지 않는지를 함께 읽어야 한다. 연구 기록은 사실, 실행 결과, 저자의 제안, 미검증 가설을 구분한다.

12. 다음 확증 실험을 어떻게 설계할 것인가

현재 결과에 모델을 계속 추가하는 방식으로 응답하지 않는다. 다음 실험의 우선순위는 물리적 블록 정체성, 지원 조건, 새로운 평가 권한이다. 새 설비나 실험 세션을 구분할 수 있다면 목표 배포 상황에 맞춰 학습·개발·교정·최종 구간을 먼저 고정한다. 독립성을 주장할 수 없는 경우에는 그 한계를 유지한다. 공동 조건이 빈약하면 어떤 외삽을 시험하는지 사전에 정의하고, 지원되는 구간만 남기는 경우 제외 분모를 공개한다.

예측기와 미래 관측 모형은 학습 안에서 적합한다. 개발 라벨의 허용 횟수, 후보 수, 선택 규칙, 계산 예산을 모든 방법에 명시한다. 현재의 사전확률 제거 중심, 학습 전용 최근접 이웃, 고정 부분집합과 강한 탐욕 기준을 유지해야 한다. 새로운 복잡한 정책에만 더 많은 교사·튜닝 자원을 주고 우수하다고 부르면 안 된다.

그다음 원형 재현과 공통 계약 어댑터를 분리한다. 예를 들어 원형의 단위 비용·정확한 B회 요청을 묶음 원천·이질 가격·보류 손실로 바꾸면 바뀐 최적화 문제를 명시한다. 예측기까지 바꾸었다면 정책 효과와 예측기 효과를 분리할 요인 대조가 필요하다. 서로 다른 논문 성능표를 같은 데이터 실측처럼 붙이지 않는다.

캐시 가설을 시험하려면 현재보다 풍부한 행동이 필요하다. 겹치는 원천, 원시 읽기 후 일부 통계만 공개, 부분 작업 뒤 실패, 재요청과 재사용을 설계한다. 모든 정책이 동일한 공개 이력과 성공·실패 정보를 받아야 한다. 이력에서 실행 상태를 복원하는 단순 기준을 포함하고, 원천 읽기와 변환 가격을 따로 기록한다. 그래야 새 상태 표현의 효과를 올바른 장부 구현의 효과와 구분할 수 있다.

종단 선택도 별도 축이다. 강제로 답하는 경우와 보류하는 경우를 같은 성능 하나로 섞지 않는다. 위험 보장을 원하면 적용할 방법의 표본 가정과 선택 절차를 실제로 충족해야 한다. 현재 문턱에 SGR이라는 이름을 붙이는 것으로 해결되지 않는다. 적응적 부분모듈성 보장 [7] 역시 해당 목적과 분포가 조건을 만족하는지 보여야 한다.

13. 예상 질문과 답변의 경계

"성능이 크게 좋아졌는데 왜 성공한 센서 정책 논문이 아닌가?" 개선은 예측기 점수식 변경에서 나왔고 한 목표에만 해당한다. 최종 벡터 오류는 여전히 약 92.9%다. 정책의 독립적 효과를 분리한 실험은 없다.

"학습 클래스가 다 있는데 왜 지원 문제라고 하나?" 목표별 클래스 존재와 네 목표 조합의 존재는 다르다. 모든 평가 헤드의 라벨은 학습에 있지만 평가 391개는 학습에 없는 공동 튜플이다. 그렇다고 공동 튜플이 없으면 예측 불가능하다고 주장하는 것도 아니다.

"새 계획을 먼저 저장했으니 평가가 블라인드 아닌가?" 현재 실행 순서는 고정했지만 같은 평가 구간을 과거에 봤다. 이를 노출된 개발 평가라고 명시하는 것이 정확하다.

"TS1이 111배 싸므로 산업 비용이 111배 개선되었나?" 6660/60이라는 견적 비율은 계산되지만 실제 전력·시간·돈의 비율은 아니다. 현재는 대리 가격에서 같은 오차라는 관찰이다.

"전체 입력 성능이 최고가 아니면 구현 버그 아닌가?" 오류 검사는 필요하지만 고정 1NN은 차원이 늘면 이웃이 바뀐다. 더 많은 정보를 무시할 수 있는 이상적 의사결정자와 다른 규칙이다. 현재 저장 예측과 집계 검증에서 그 악화가 확인되었다.
"왜 수십만 예측으로 통계적 유의성을 안 구하나?" 실제 평가 사이클은 421개이며 물리적 독립 단위가 불명이다. 같은 사이클의 여러 모델·마스크·정책 행은 새 독립 표본이 아니다.
"선택 부분집합이 평가에서 졌으면 다른 동률 규칙으로 고치면 되지 않나?" 후속 연구의 후보로 만들 수 있지만 같은 평가 결과를 본 뒤 고친 규칙을 최초 선택 성능으로 제시할 수는 없다. 현재 실패를 보존하고 별도 새 평가가 필요하다.
"이 연구의 새 기여가 너무 절차적인 것 아닌가?" 현재 실증 기여는 지원·예측기·종단·정책을 분리한 감사와 구체적인 반례다. 새 정책의 독립 성능 기여는 아직 없다. 더 강한 알고리즘 논문이 되려면 위 확증 실험에서 단순 기준을 넘는 효과를 보여야 한다.
"실제 교수들의 지도를 받은 결과인가?" 이번 실험·문헌 근거는 공개 논문과 로컬 실행 기록이다. Research Forge의 인물 사진, 철학 렌즈, UI 해설을 직접 인간 교수의 검토나 현재 실험의 추가 증거로 승격하지 않는다.

14. 재현 부록을 읽는 순서

먼저 v12 PLAN.json과 실행의 locked_plan.json을 읽는다. 후보·마스크·선택 권한·노출 경고가 무엇인지 확인한다. 이어 source_receipt.json의 원시 파일과 특징 추출 의존성, consumed_code/run_experiment.py의 실제 바이트를 본다. 현재 디렉터리의 파일 이름만 같다는 것으로 당시 코드가 같았다고 가정하지 않는다.
events.json은 계획 고정, 학습·선택 시작, 모델·정책 고정, 현재 평가 디코딩 완료의 순서를 담는다. frozen_fit.json은 학습으로 적합한 매개변수, frozen_selection_and_policies.json은 선택된 모델·부분집합이다. 저장된 선택·평가 JSONL은 각 사이클의 예측과 라벨을 대조할 수 있게 한다. results.json은 이를 집계한 것이다. v12 verify.py는 새 예측 호출 없이 개수·오차·지원·견적을 재계산한다.
v12 원본 실행 영수증에는 소비 코드가 읽은 v9 계획 파일 자체의 해시가 빠져 있었다. 원본 영수증은 고치지 않았다. 최종 VALIDATION.json에서 현재 v9 계획 SHA를 v11의 동결 영수증 해시와 대조해 추가로 결박한 사실을 남겼다. 원래부터 완벽했던 기록처럼 보이게 바꾸지 않는다. 이 조치는 수치를 변경하지 않으며, 해시 연결은 과학적 독립성이나 서명 검토를 뜻하지 않는다.
기존 실행을 다시 쓰지 않는다. 재현이 필요하면 run_experiment.py --out에 새 출력 폴더를 준다. 이번 v13은 재실험을 하지 않고 기존 해시·결과·그림 수치를 점검한다. 기존 v12의 여덟 의미 있는 계약 검사는 학습 전용 적합, 비공개 차원 배제, 사전확률 항의 통제, 동률·선택, 예산·단위를 확인했다. 실제 데이터 결과와 이 작성된 검사 fixture를 같은 성능 표본으로 세지 않는다.

15. 출처 장부와 주장 장부의 역할

SOURCES.json에는 기존 18개와 v14 추가 문헌 1개를 합친 공식 1차 페이지 19개와 로컬 근거 파일의 해시가 있다. 공식 초록만 읽은 항목은 그대로 표시한다. HTML의 일부 정의와 방법만 읽은 항목도 범위를 적는다. 원 HTTP 응답 바이트를 저장하지 않았으므로 출판사 원문 바이트 해시는 null이다. 대신 URL·확인 범위·시각·요약의 정규 JSON에 대한 review_record_sha256을 기록한다. 이것을 원 논문 파일 해시로 읽으면 안 된다.
CLAIMS.json은 실제 실행 결과, 기술적 파생, 문헌의 제한된 설명, 저자의 제안을 구분한다. 예를 들어 25%포인트는 실행 집계에 연결되고, 연구 순서를 개선하자는 제안은 그와 다른 증거 유형이다. 미래 확증 실험은 현재의 지원된 결론 목록이 아니다. 그림도 저장 결과에서 나온 수치만 표현하며 새 실험을 만든 것이 아니다.
공부할 때는 영어 본문의 질문 → 대응 그림 → 원 결과 키 → 반대 사례 → 다음 필요한 근거 순서로 읽으면 된다. 최종적으로 기억할 것은 "성능을 올린 모델 이름" 하나가 아니라, 어느 분모에서 무엇이 좋아졌고, 어느 분모가 비어 있으며, 어떤 비교가 아직 수행되지 않았는가다.

16. v14가 실제로 새로 계산한 질문

v13까지는 고정 예측기의 전체 오차가 좋아져도 복잡한 획득 정책의 승리라고 부를 수 없음을 확인했다. v14는 거기서 한 걸음 더 들어간다. 같은 예측기와 같은 정보를 놓고, 무엇을 성공으로 정의하

는지 바꾸면 실제로 다른 정보를 선택하는가? 그 선택은 어떤 부품을 돕고 어떤 부품을 희생하는가? 완전한 상태 진단 자체가 현재 예측기의 출력 방식 때문에 막혀 있는가?
이를 확인하려고 예측기를 추가하지 않았다. 이미 selection에서 고정했던 1NN과 세 채널의 8개 mask 예측을 그대로 쓴다. 변경한 것은 mask를 고르는 선택 목적이다. 평균 성분 오류, 완전 벡터 오류, 최악 부품의 집계 오류, 펌프 오류 네 가지를 새 계획에 고정했다. raw 가격의 5개 예산과 equal 가격의 4개 예산에서 각 목적이 하나의 mask를 고른다.
새 표의 36행은 4개 목적×9개 가격·예산 계약이다. 이전 표의 36행은 4개 정책 기준×9개 계약이었다. 숫자가 같다고 같은 분석을 새 성능 실험처럼 다시 센 것이 아니다. 새 분석은 선택 목적을 바꾸는 명시적 의사결정 대조다. 그러나 421개 사이클의 저장 예측을 재사용하므로 독립 표본·새 예측·새 현장 실험도 아니다. 후속 계산을 계획할 때 저자는 이미 과거 집계 결과를 알고 있었다. 이를 사후 개발 분석이라고 부르는 이유다.

17. 네 손실은 모두 비율이지만 같은 성공이 아니다

사이클 i와 부품 h의 오류를 e(i,h)=0 또는 1이라고 하자. 평균 성분 오류 C는 모든 오류를 더하고 4n으로 나눈다. 벡터 오류 V는 네 부품 중 하나라도 틀린 사이클을 세고 n으로 나눈다. 최악 부품 오류 W는 부품별 n개 판정 오류율 네 개를 계산한 다음 그중 가장 큰 값을 취한다. 펌프 오류 P는 세 번째 목표만 n개 사이클에서 센다.
여기서 흔한 혼동이 있다. 사이클마다 네 오류의 최댓값을 구해 평균하면 그것은 벡터 오류 V다. v14의 W는 먼저 부품별 전체 사이클을 집계한 뒤 최댓값을 구한다. 연산 순서가 다르기 때문에 W와 V는 같은 지표가 아니다. 코드의 worst_head는 max(head_error_counts)/n, pump는 head_error_counts[2]/n이다.
각 사이클의 틀린 부품 수를 k라고 하면 k/4 ≤ 1(k>0) ≤ k다. 이를 평균하면 C ≤ V ≤ min(1,4C)가 나온다. C가 작다는 사실은 V도 같은 정도로 작다는 뜻이 아니다. 예를 들어 세 사이클 중 한 사이클의 네 부품을 모두 틀리는 규칙은 C=1/3, V=1/3이다. 반대로 세 사이클에서 하나씩만 틀리면 C=1/4, V=1이다. 평균 성분으로는 두 번째가 좋지만 완전 진단으로는 첫 번째가 좋다. 이 예는 합성 경계검사이며 실제 유압 성능표에 합치지 않았다.
loss별 의사결정 차이는 새로운 일반 이론이라고 주장하지 않는다. 추가한 1차 문헌 [19]는 이 구분의 선행 논의를 확인하는 비교 근거다. 해당 문헌의 이진 multilabel 설정과 이번 다중 클래스 네 목표를 같은 정리의 적용 사례라고 단정하지 않았다.

18. 선택 단계는 어떻게 고정했는가

각 목적은 204개 selection 사이클에서 자기 손실과 0.05×quote/전체 원천 quote의 합을 최소화한다. 먼저 hard budget 안에 들어오는 mask만 남긴다. 정확히 같은 목적값이면 견적이 작은 것, 그래도 같으면 mask 번호가 작은 것을 택한다. 작은 부동소수점 오차가 동률을 깨지 않도록 Fraction으로 분수 비교를 했다.
목적이 달라도 계수 0.05를 썼지만, 이는 네 오류의 산업 가치를 똑같다고 조사했다는 뜻이 아니다. 같은 숫자의 비용 계수에서 선택이 어떻게 달라지는지 보는 선언된 민감도 계약이다. raw의 표본 수 대리 견적과 equal의 채널 수 견적도 별개다. 이 둘의 60과 1을 합치거나 동일한 돈으로 환산하지 않는다.
실행 전에 새 폴더에 PLAN, runner, selection/evaluation prediction JSONL, profile, 기존 고정 선택 및 결과를 복사하고 해시를 남겼다. 실제 계산은 복사한 runner subprocess가 했다. 선택 함수는 selection 집계만 인자로 받는다. 그 36개 결정을 저장한 다음 현재 evaluation 집계를 계산했다. 다만 분석 장치가 이미 노출된 profile 라벨을 가지고 있고 저자도 과거 결과를 알고 있으므로 이 순서가 블라인드 평가를 되살리지는 않는다.

19. 벡터 목적을 넣었는데 선택이 하나도 바뀌지 않은 이유

새 계산에서 component와 vector 목적의 mask는 9개 계약 모두 같았다. 반면 pump 목적은 7개, worst-head 목적은 2개에서 달라졌다. 목적함수를 바꾸면 언제나 행동이 달라진다는 인상적인 이야기보다 이 무변화가 먼저 나와야 한다.

실제 selection에서 1NN의 모든 비어 있지 않은 mask는 벡터 오류가 100/204로 같았다. 벡터 목적 안에서는 정보 조합을 더 잘 구별할 신호가 없었다. 그래서 기존의 견적·mask 번호 동률 규칙이 그대로 선택을 정했다. 이는 component와 vector 손실이 일반적으로 같다는 증거가 아니다. 앞 절의 반례처럼 다를 수 있지만 이 고정 예측기·자료에서는 선택에 필요한 차이가 포화되어 있었다.
raw 예산 60에서 pump 목적은 아예 빈 mask를 택했다. 그 예산에 허용되는 TS1이 펌프 예측을 바꾸지 않으므로, 같은 펌프 오류에 quote를 줄이는 쪽이 낫기 때문이다. 다른 부품의 성능과 완전 진단은 악화된다. 이를 "아무 정보도 필요 없다"는 일반 결론으로 옮기면 안 된다. 어떤 목적을 고정했는지가 결론의 일부다.

20. 펌프를 도우면 누가 대가를 치르는가

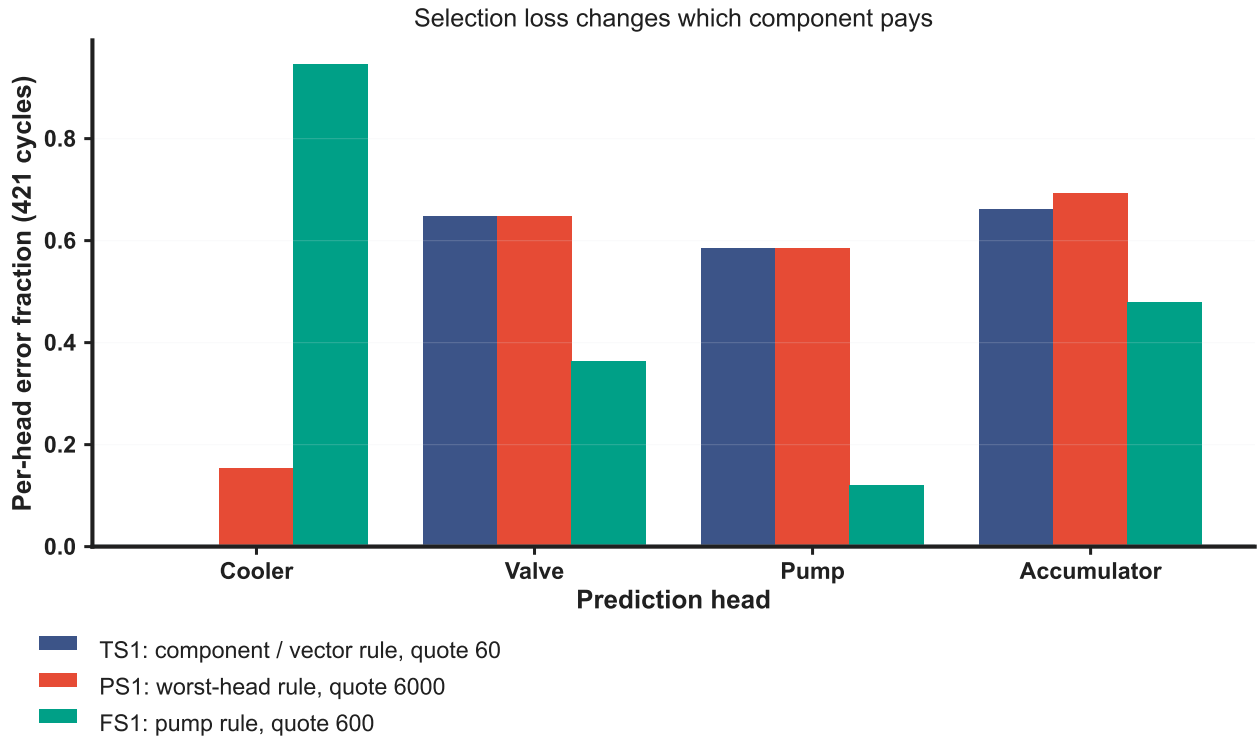


그림 8: 새 그림 4: 목적에 따른 부품별 오류 교환

raw 예산 6660에서 component/vector는 TS1(quote60), worst-head는 PS1(quote6000), pump는 FS1(quote600)을 고른다. 이 세 선택을 같은 421개 evaluation 사이클과 같은 head 분모로 펼치면 평균 한 숫자에서 가려진 교환이 보인다.

선택	cooler 오류	valve 오류	pump 오류	accumulator 오류	벡터 오류
TS1: component/vector 목적	0	273	246	279	391
PS1: worst-head 목적	65	273	246	292	391
FS1: pump 목적	398	153	51	202	408

각 head 열과 벡터 열의 분모는 모두 421이지만 head 오류와 완전 벡터 오류의 사건은 다르다. FS1은 펌프 오류를 246개에서 51개로 크게 줄인다. 그러나 cooler 오류는 0개에서 398개, 완전 벡터 오류는 391개에서 408개로 늘어난다. 최악 부품 오류도 279/421에서 398/421로 악화된다. 평균 성분 오류만 보면 798/1684와 804/1684의 작은 차이지만, 어떤 부품을 틀리는지는 완전히 다르다.
이것은 실제 펌프 안전성을 입증한 결과가 아니다. 작업 손실과 사고 비용은 조사하지 않았고 같은 노출 기록의 오류를 다른 우선순위에서 계산했을 뿐이다. 따라서 "pump 목적이 더 좋다"가 아니라 "pump 목표에 유리한 선택이 다른 목표와 충돌한다"가 정확한 해석이다.

21. 좋은 목적을 골라도 선택이 미래 구간으로 옮겨가지는 않는다

worst-head 목적은 raw 예산 6000과 6660에서 PS1을 골랐다. 하지만 evaluation의 최악 부품 오류는 PS1 0.693587, 훨씬 싼 TS1 0.662708이었다. 견적은 각각 6000과 60이다. 이 경우 TS1은 네 손실 모

두에서 PS1보다 나쁘지 않고 비용도 적다. 따라서 펌프라는 숨은 목적을 빼놓아서 생긴 역전이라고 설명할 수도 없다. 선택 자료의 목적 최적화가 다음 노출 구간에서 실패한 구체적 반례다.
여기에 hindsight regret를 붙인다. 같은 목적·같은 가격·같은 budget에서 evaluation 라벨을 보고 고른 최선의 고정 mask를 기준으로, 선택 규칙의 손실+견적 항이 얼마나 높은지 뺀 값이다. worst-head PS1의 이 후회는 약 0.075473이다. equal 가격의 양의 예산에서 component가 고른 PS1은 TS1 대비 0.046318의 component 후회가 있다.
반대로 vector 목적은 같은 PS1에서 vector 후회가 0이다. PS1과 TS1의 vector 오류와 quote가 같기 때문이다. 서로 다른 목적의 후회를 섞어 하나의 우열표를 만들면 안 된다. hindsight 기준 자체도 evaluation 정답을 봤으므로 배포 가능한 선택기나 독립 확인 성능이 아니다. 그 낙관적인 최선보다 얼마나 부족한지를 보는 진단일 뿐이다.
Pareto도 같은 주의가 필요하다. v14의 projection은 component 오류, vector 오류, quote 세 좌표만 본다. 이 공간에서 36행 중 17행이 엄격히 지배된다. 이는 예산 계약에 반복된 행 수이며 17개의 독립 실패 사건이 아니다. 특히 pump가 고른 FS1은 TS1에 이 projection으로 지배될 수 있지만 펌프 오류는 더 낮다. Pareto에서 빠진 목적을 숨긴 채 보편적으로 열등하다고 결론 내릴 수 없다.

22. 가장 중요한 구조: 예측기가 만들 수 없는 완전 벡터

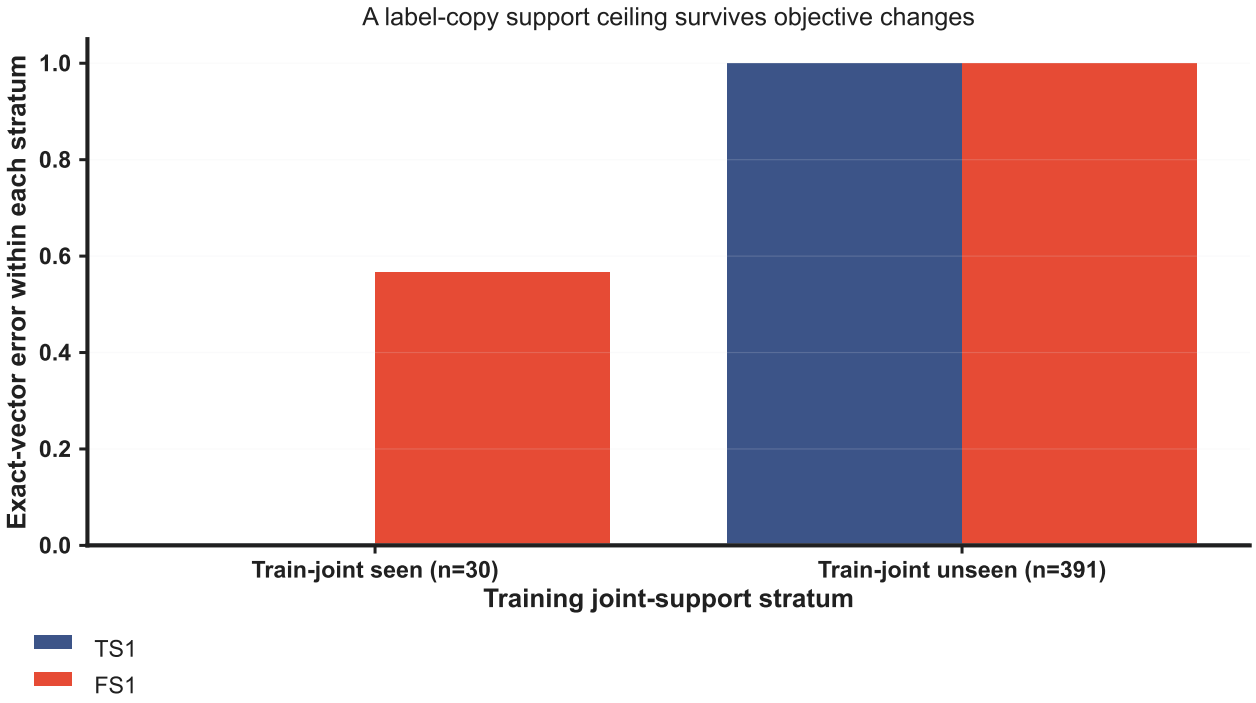


그림 9: 새 그림 5: 학습 조합 지원과 완전 벡터 오류

T를 학습에 존재하는 네 라벨 tuple의 집합이라고 하자. 비어 있지 않은 mask의 현재 1NN은 가장 가까운 하나의 학습 사이클을 골라 그 사이클의 라벨 네 개를 통째로 복사한다. 따라서 출력은 반드시 T 안에 있다. 참 tuple이 T 밖인 사이클에서는 어떤 이웃을 선택하든 참값과 완전히 같아질 수 없다. 각 미지원 사이클의 정확 벡터 성공은 0이므로, 전체 성공 수의 상한은 참 tuple이 T 안에 있는 사이클 수다. 표본 독립성이나 확률 모형 없이 성립하는 유한 집합 논증이다.
여기에는 빠뜨리기 쉬운 예외가 있다. 빈 mask의 출력은 head별 최빈값을 조합한 fallback이다. 각 최빈 라벨이 학습에 있더라도 그 조합이 T 안에 있을 필요는 없다. 예를 들어 학습 tuple이 (1,0,0,0), (0,1,0,0), (0,0,1,0)이면 head별 최빈 조합 (0,0,0,0)은 학습에 없다. 따라서 일반적인 modal fallback에 training-support 상한을 자동 적용하면 틀린다.
실제 이번 자료에서는 mask0의 fallback (3,100,0,90)이 T 안에 있음을 별도로 확인했다. verifier는 두 구간 합계 5,000개 저장 1NN 행에서 이 조건을 검사했다. 비어 있지 않은 mask 행 4,375개와 빈 mask 행 625개를 구분했으며 이는 새 예측 횟수나 독립 사이클 수가 아니다.

evaluation에서는 T 안의 참 tuple이 30개, T 밖이 391개다. 실제 모든 8mask가 미지원 391개에서 벡터 오류 100%였다. 이것이 cohort에서 직접 확인한 수치다. 위 출력 제한을 전제로 하면 정확 벡터 성공 상한은 30/421=7.1259%, 벡터 오류 하한은 391/421=92.8741%다. TS1은 지원된 30개를 모두 맞혀 이 상한을 달성한다. 단, raw budget이 최소 60, equal budget이 최소 1일 때만 TS1을 요청할 수 있다. budget0에서는 빈 mask만 허용되고 421개 전부를 틀리므로 상한이 달성되지 않는다.
FS1은 지원된 30개에서도 17개 벡터를 틀린다. 미지원 391개를 더하면 전체 벡터 오류가 408개다. 반면 같은 미지원 구간의 펌프 오류는 TS1 246/391, FS1 51/391로 다르다. 완전 벡터의 성공이 막혔다고 부품별 정보가 아무 가치가 없다는 뜻은 아니다. 포화된 완전 진단 지표와 부분 진단 효용을 같이 보아야 한다.
이 상한은 1NN과 같은 학습 tuple 복사 출력군에 관한 조건부 결론이다. head별 독립 centroid는 학습에 없던 조합을 새로 만들 수 있으므로 전제가 다르다. 새로운 representation·decoder도 마찬가지다. 실제 적응형 정책을 평가한 것이 아니며, 상한의 집합 논증을 적응형 정책 효과 실측으로 바꾸지 않는다. 보류로 coverage를 0으로 만들면 벡터를 맞힌 것도 아니다. 기존 v11의 빈 selective-risk 분모는 여전히 null이다.

23. 이 결과 뒤에 필요한 확증 실험

첫째, 목표 손실을 정하는 주체와 작업 가치를 확보해야 한다. component·vector·pump 중 어떤 손실을 줄일지 임의로 바꾼 뒤 산업 성능이라고 할 수 없다. 실제 업무가 전체 상태 벡터를 요구하는지, 일부 부품 경보가 가치를 갖는지, 놓침과 오경보의 비용이 무엇인지를 먼저 정한다.
둘째, 새 출력 방식을 검증해야 한다. training tuple 복사에서 head별 출력 조합이나 조건 모형으로 바꾸면 이번 상한의 전제는 깨진다. 그것은 상한이 틀렸다는 뜻이 아니라 다른 가설군을 시험한다는 뜻이다. 같은 학습·선택 권한과 head별 오류·벡터 오류를 유지하고 강한 prior-neutral centroid·고정 부분집합 기준을 남겨야 한다. 새로운 예측기의 효과와 새로운 획득 정책의 효과는 별도 요인이다.
셋째, 새 장치·세션 정체성과 지원되는 조건 조합, 새 evaluation 권한이 필요하다. 이번에 selection 규칙을 먼저 저장했어도 evaluation 자료는 예전부터 노출되었고 물리적 독립 단위는 불명이다. 목적을 계속 추가해 같은 421개에서 잘 되는 것을 고르는 방식은 후속 개발일 수는 있어도 최종 확인이 될 수 없다.
넷째, 실제 adaptive 방법을 비교하려면 미래 관측·정보 공개·훈련·교사·튜닝·계획 비용을 맞추고 새 rollout을 실행해야 한다. 여기의 최적 hindsight mask는 라벨을 봤고, Pareto는 사후 평가 좌표다. 둘을 실제 정책으로 가져올 수 없다. 부분 공개·캐시 겹침·실패 후 작업 재사용처럼 아직 구현하지 않은 실행 상태 가설도 별도의 실험으로 남긴다.

24. 재현 명령과 예상 반론

python -X utf8 -B projects/research_advancement_v14_20261019/probeflow/test_experiment.py
python -X utf8 -B projects/research_advancement_v14_20261019/probeflow/verify.py --run runs/research_advancement_v14_20261019_01/probeflow
검증은 보존 결과를 읽기만 한다. 새 계산이 필요하면 run_experiment.py --out에 이전에 존재하지 않는 새 폴더를 준다. 실제 입력과 코드 복사 후 그 소비 코드로 실행하며 기존 결과는 덮어쓰지 않는다. 경계 fixture 8개와 수식 13개의 실제 Typst 컴파일도 구분해 기록했다. fixture는 유압 데이터 실험의 표본 수에 들어가지 않는다.
"vector 목적을 넣었는데 변화가 없으면 v14는 실패 아닌가?" 목적과 행동이 달라질 것이라는 가설에는 부정 결과다. 그러나 모든 nonempty mask의 선택 vector loss가 같고 출력 지원 상한이 있다는 원인을 정량적으로 밝힌다. 이 실패를 숨기고 새 목적을 계속 추가하지 않았다.
"상한은 단순한 1NN 성질 아닌가?" 집합 논증 자체는 간단하다. 새로운 보편 정리라고 주장하지 않는다. 중요한 것은 현재 구현이 실제로 그 출력군에 속하는지, 빈 mask 예외가 있는지, 예산 안에서 상한이 달성되는지, 미지원 구간에서 어떤 지표가 포화되는지를 실행 기록에 연결한 점이다.
"펌프 정확도가 크게 좋아졌는데 그대로 쓰면 안 되나?" 다른 부품과 완전 진단이 악화되고 데이터는 한 장치의 노출 구간이다. 실제 작업 손실·검증된 독립 단위·새 평가가 없다. 특정 head 오류 개선은 작업 효용·현장 안전성·일반화를 대신하지 못한다.

"hindsight regret가 0이면 선택이 검증된 것 아닌가?" 같은 노출 구간과 제한된 8mask 중, 선언한 손실에 대해 사후 최선과 동률이라는 뜻이다. 최선 자체가 가능한 작은 후보 집합·노출된 조건·가중치에 의존한다. 독립 평가나 더 강한 predictor·정책과의 비교가 아니다.

Acknowledgements

This work was created as part of the No-More-Slops group activities at Pseudo Lab.